

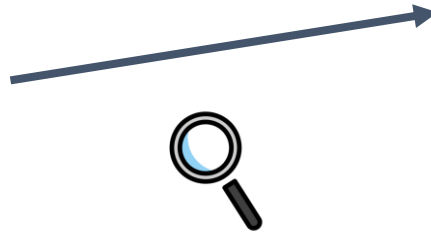
Apuntes clase 5

Planificación de procesos

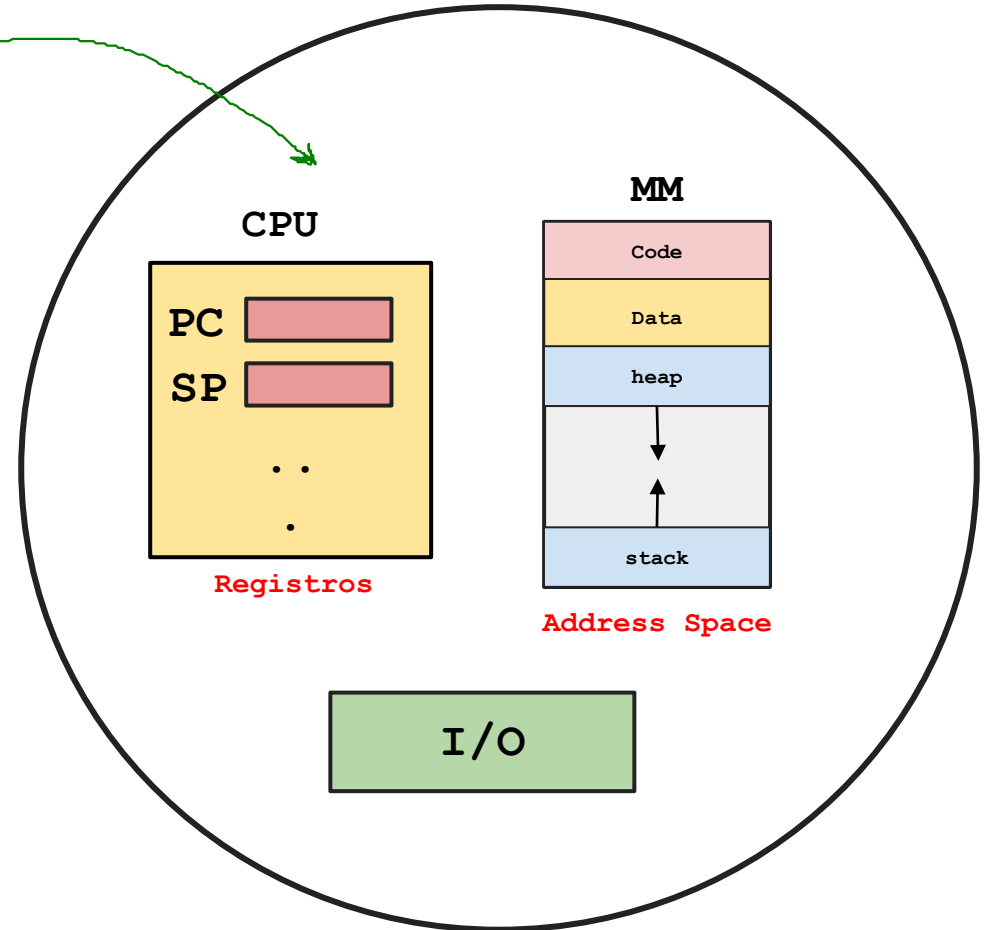
03/03/2026

Procesos

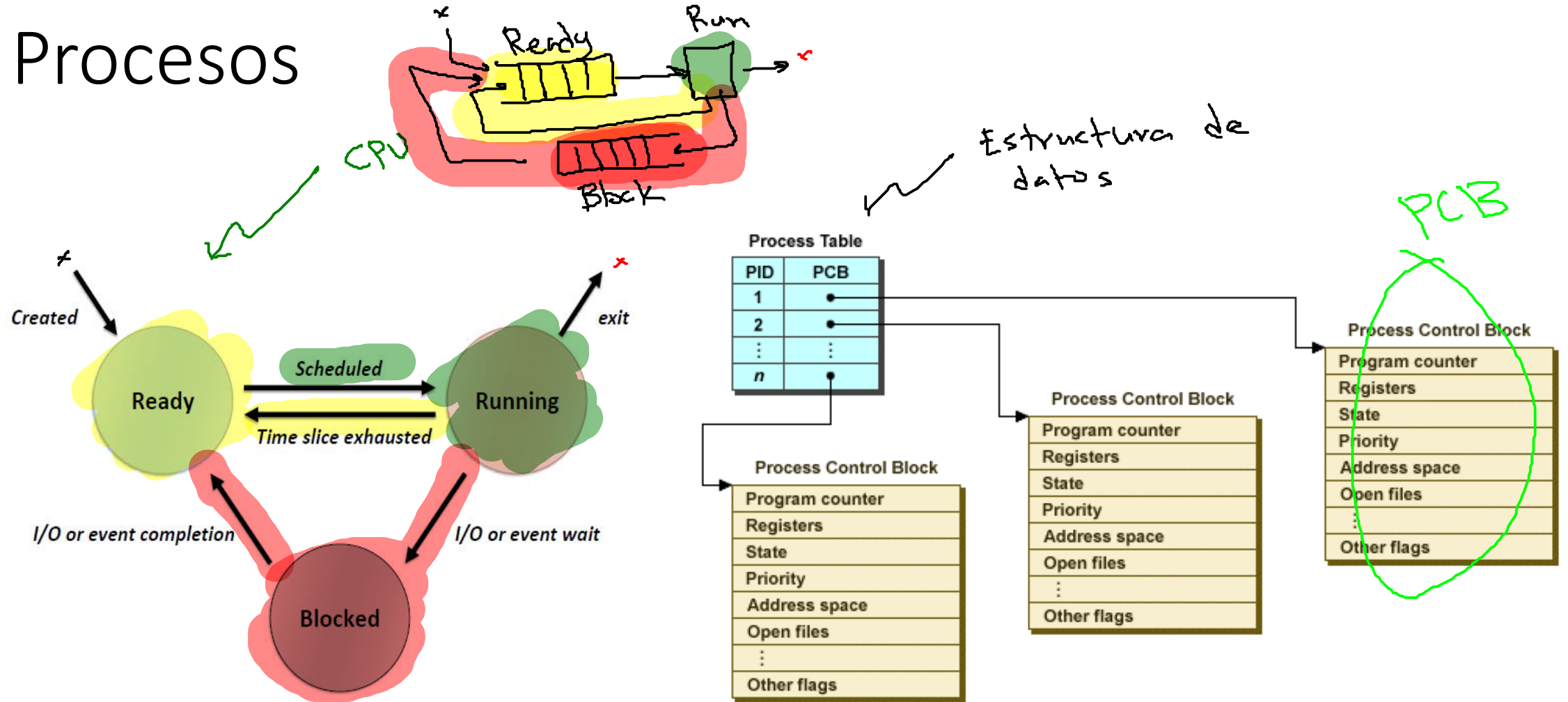
Proceso = CPU + Memory + I/O Info



Abstracción

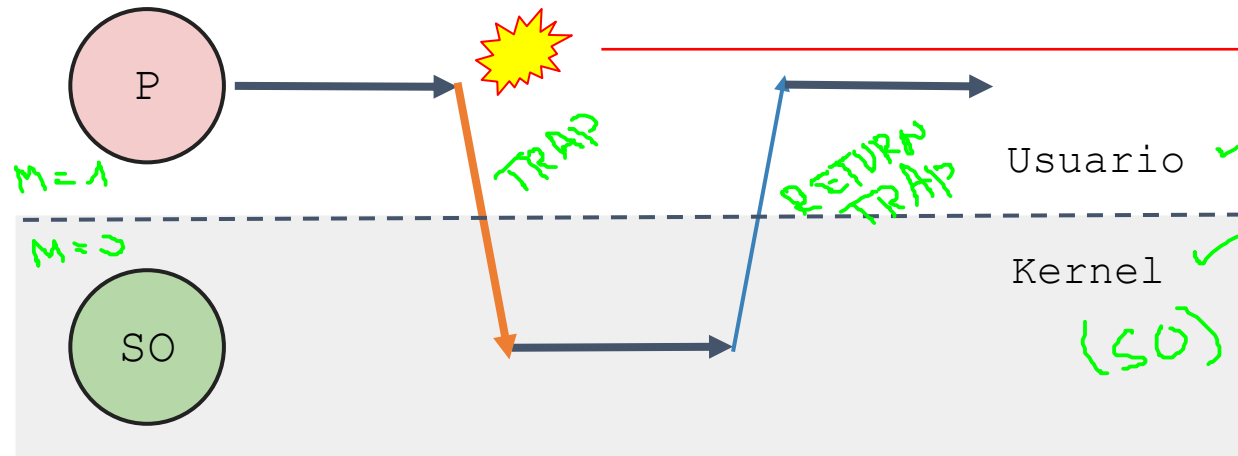


Procesos



Ejecución directa limitada

✓ **Limited**



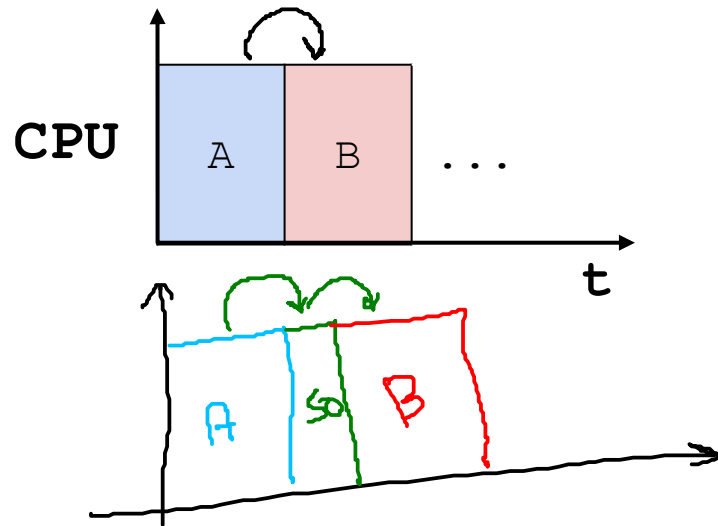
Eventos:

- Excepción ✓
- Syscall ✓
- Interrupción ✓

✓ **Direct Execution**



Aproximaciones – Cambio de contexto



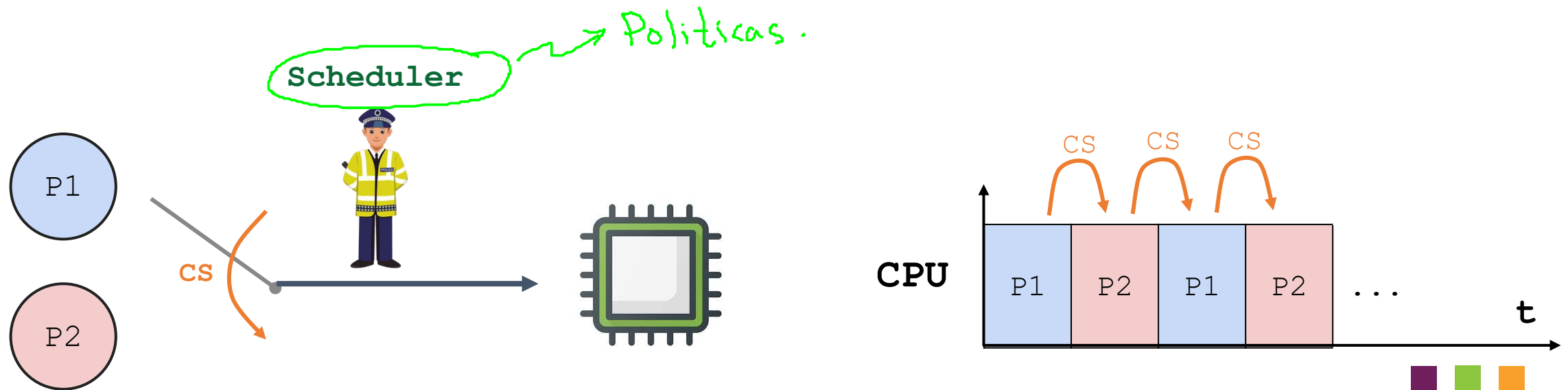
Propuestas

- ✓ **1. Cooperativa:** El proceso cede la CPU al SO.
 - yield.
 - Excepción.
- ✓ **2. No cooperativa:** El SO toma el control de la CPU
 - Interrupciones
 - timer

Contextualización

Cómo desarrollar una política de Scheduling

- ¿Cómo desarrollar un framework básico para tratar con las políticas de scheduling? ✓
- ¿Qué suposiciones claves usar? ✓
- ¿Cuáles son las métricas más importantes? ✓
- ¿Cuáles son los enfoques básicos utilizados en los primeros sistemas informáticos? ✓

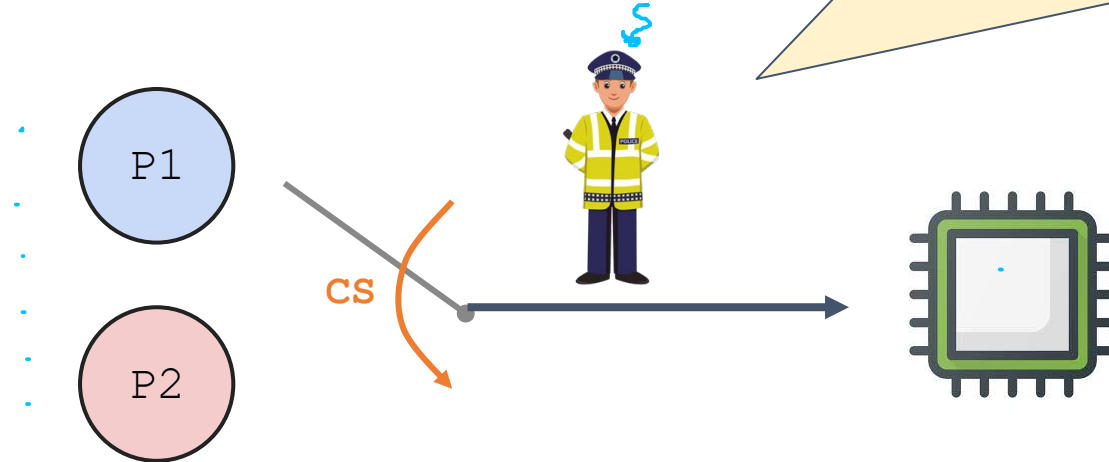


Contextualización



Algunas preguntas claves:

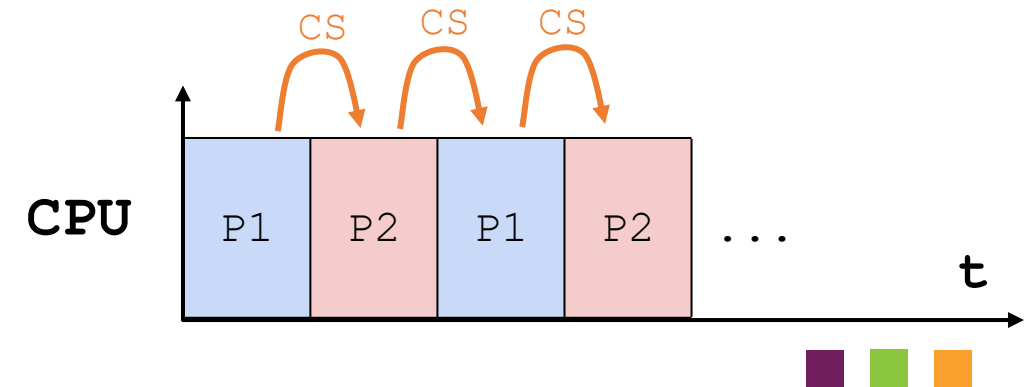
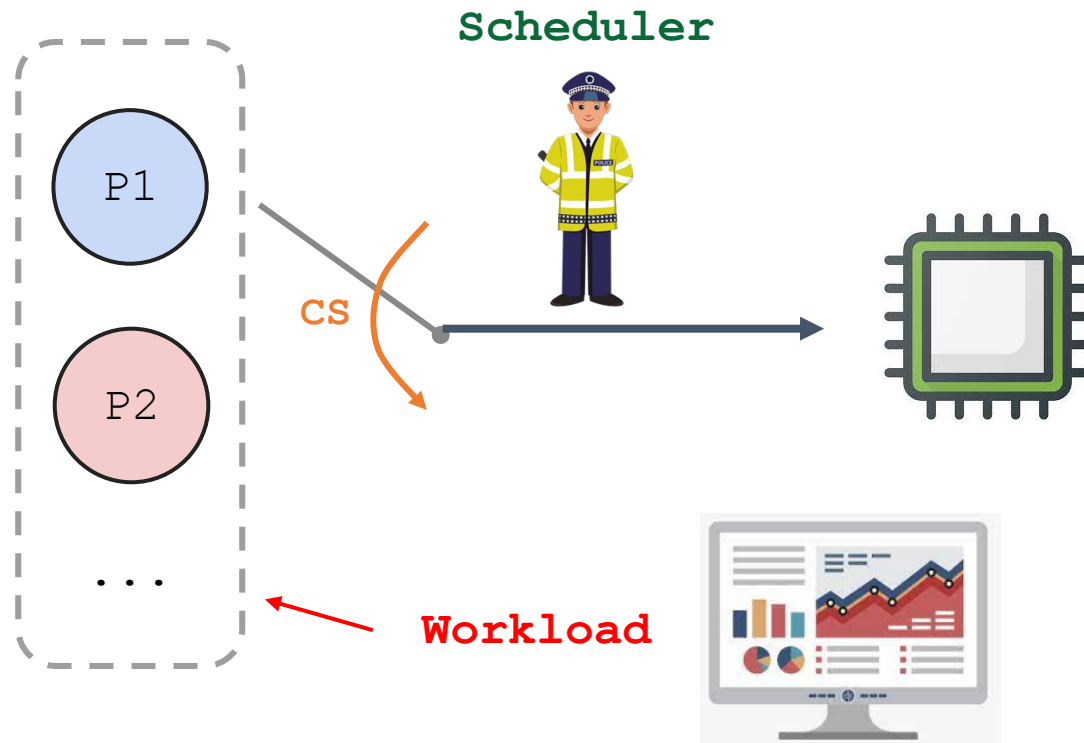
- ¿Qué proceso elegir? ✓
- ¿Teniendo en cuenta que aspectos? → Métricas ✓



Suposiciones de carga de trabajo

Conceptos importantes:

- **Scheduler (planificador)**: Decide como y cuando los procesos acceden a la CPU (o CPUs compartidas) de acuerdo a unas **métricas**.
- **Workload (carga de trabajo)**: Procesos en ejecución en un sistema.



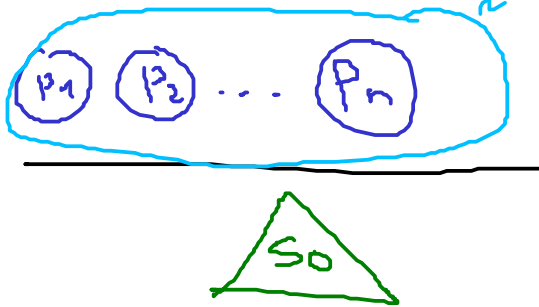
Suposiciones de carga de trabajo

→ Ideal → Realidad

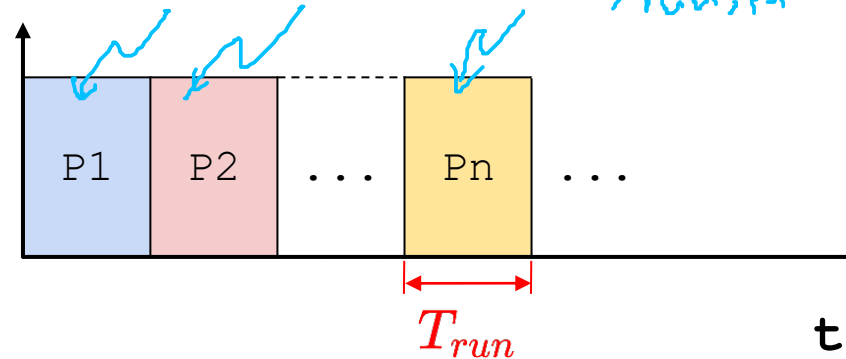
Suposiciones iniciales: Son “ideales y poco realistas”.

1. Cada trabajo se ejecuta por la misma cantidad de tiempo.

Workload (carga de trabajo)



CPU

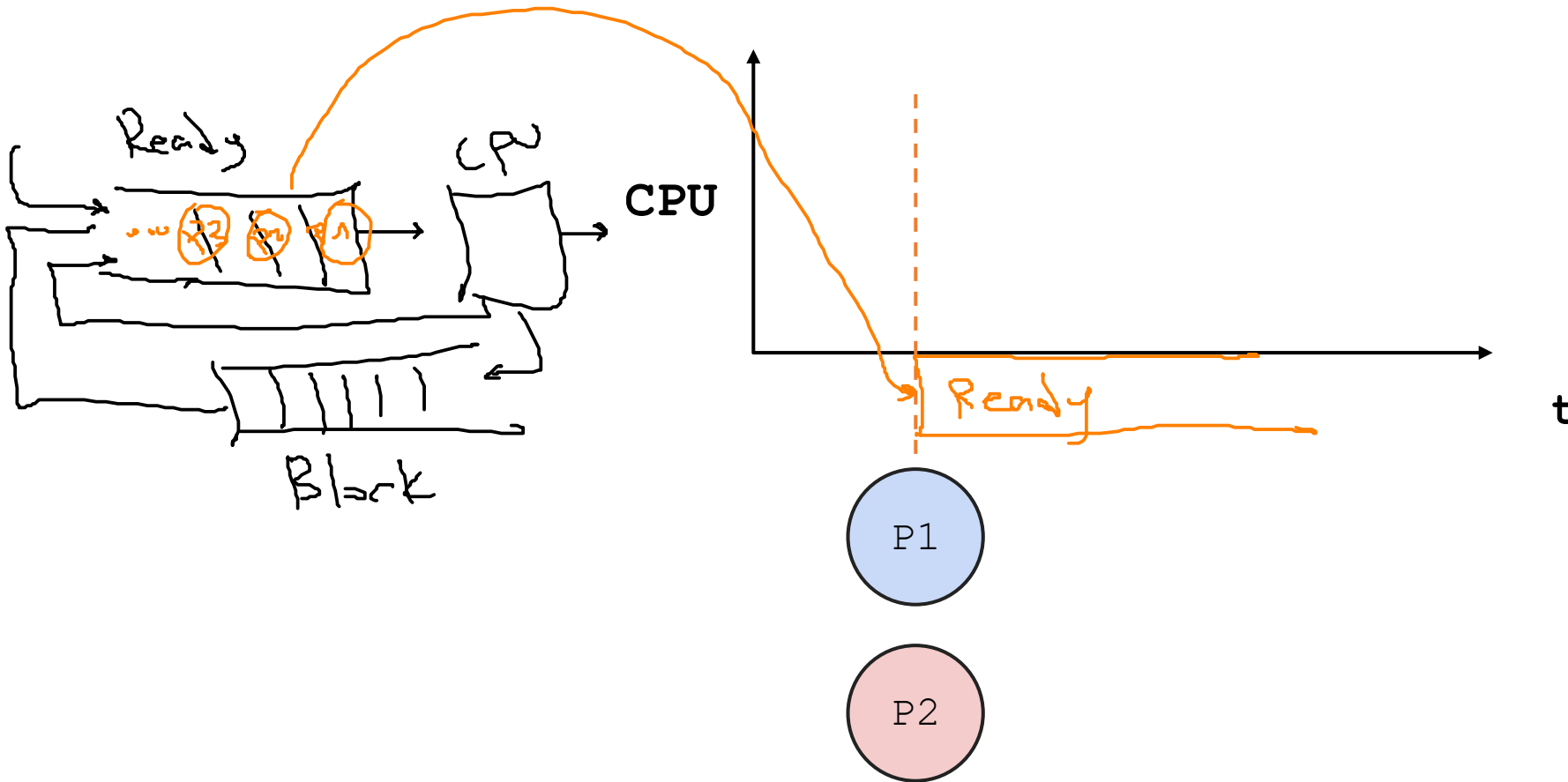


$$T_{run, P_1} = T_{run, P_2} = \dots$$

Suposiciones de carga de trabajo

Suposiciones iniciales: Son “ideales y poco realistas”.

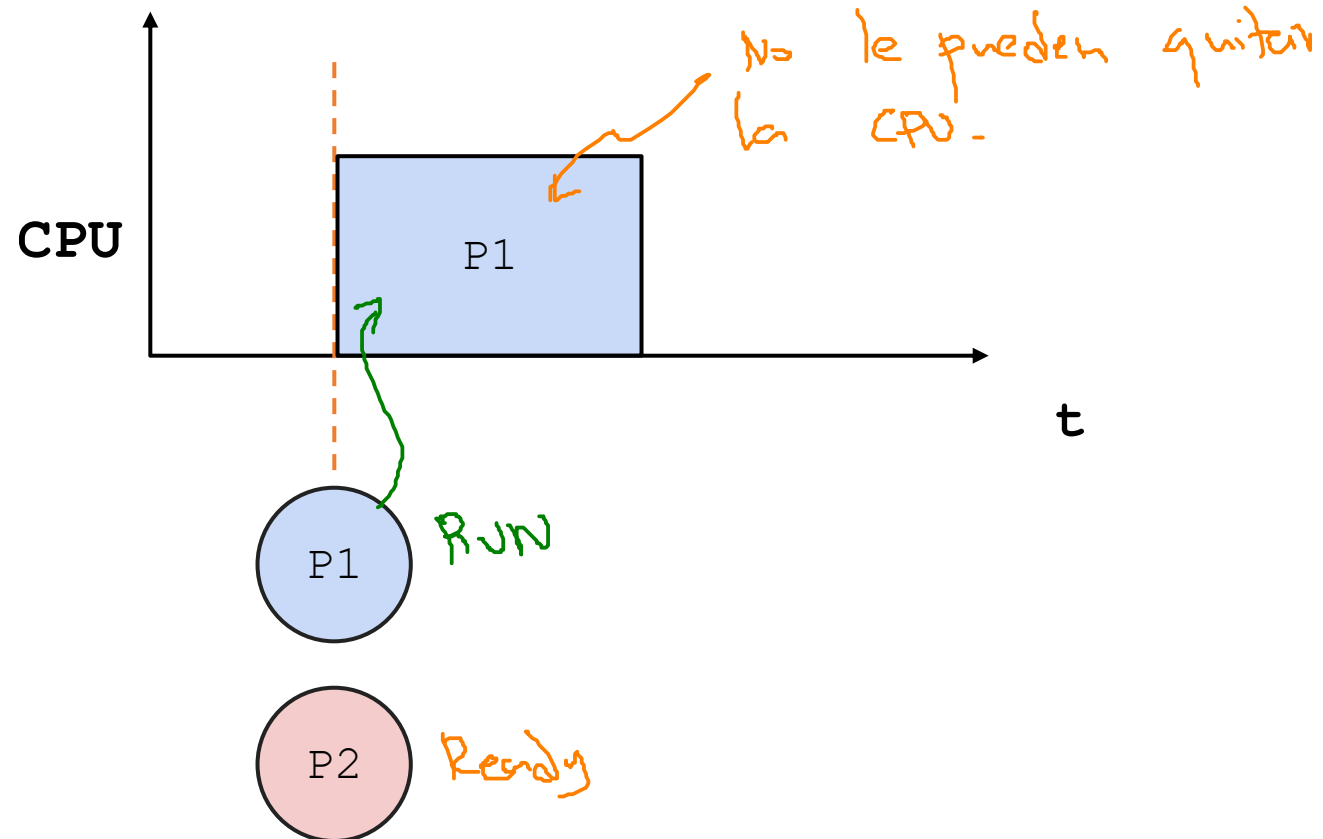
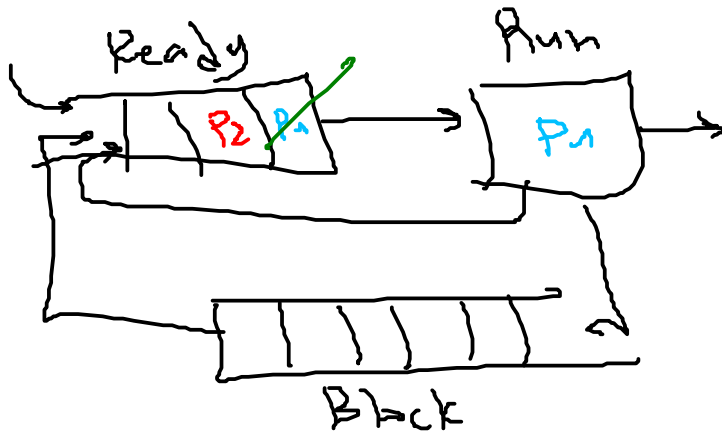
2. Todos los trabajos son iniciados al mismo tiempo (arrival time).



Suposiciones de carga de trabajo

Suposiciones iniciales: Son “ideales y poco realistas”.

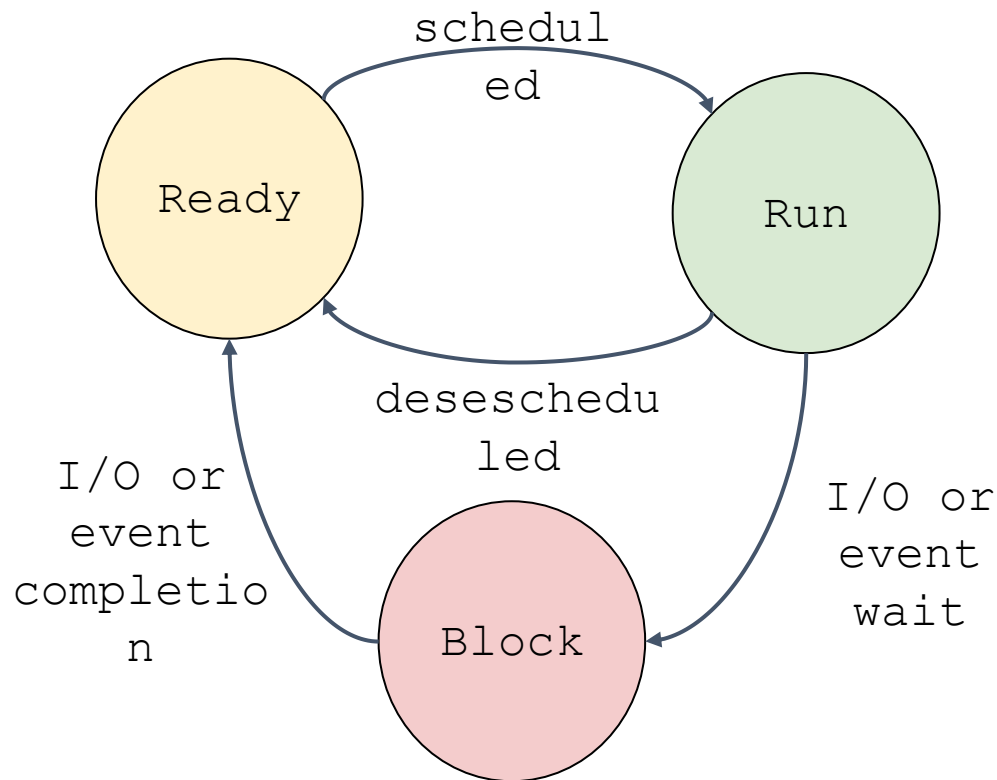
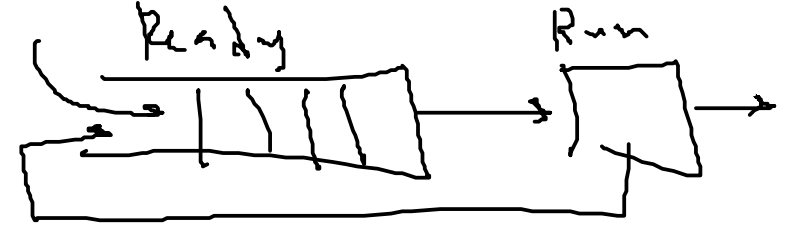
3. Una vez iniciado, cada trabajo se ejecuta hasta su finalización (no se puede interrumpir).



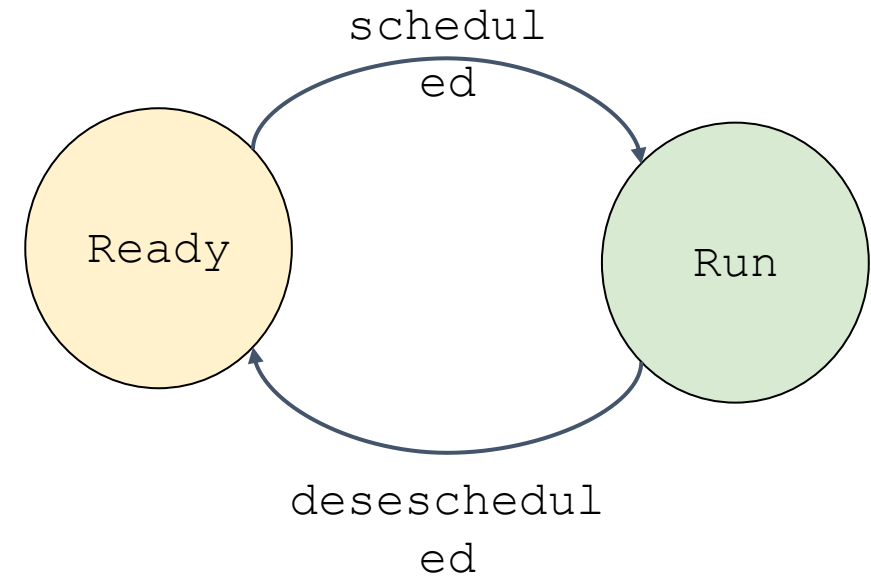
Suposiciones de carga de trabajo

Suposiciones iniciales: Son “ideales y poco realistas”.

4. Todos los trabajos solo usan la CPU (no I/O).



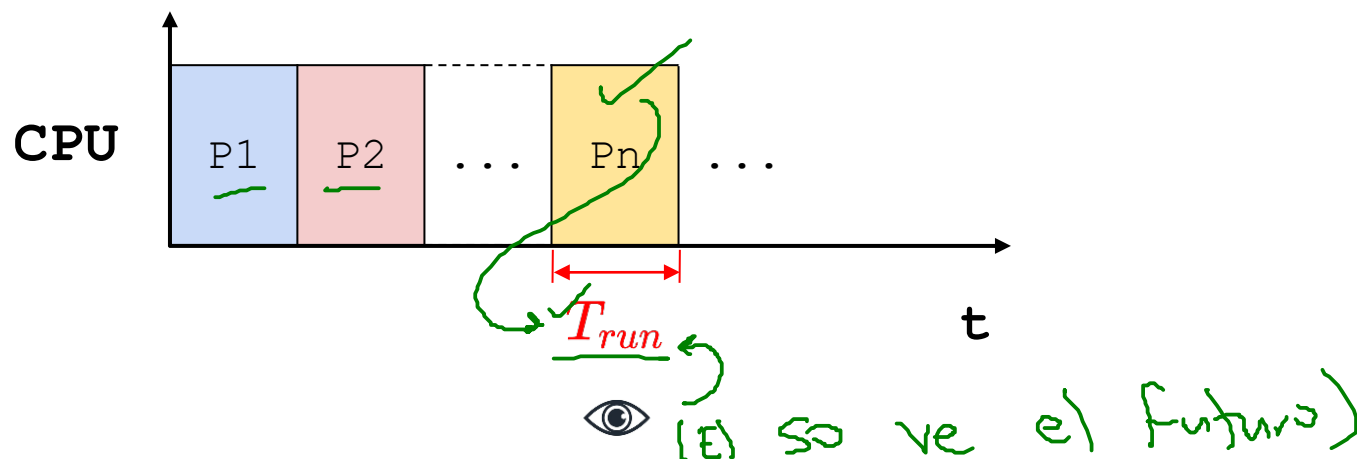
Simplificación



Suposiciones de carga de trabajo

Suposiciones iniciales: Son “ideales y poco realistas”.

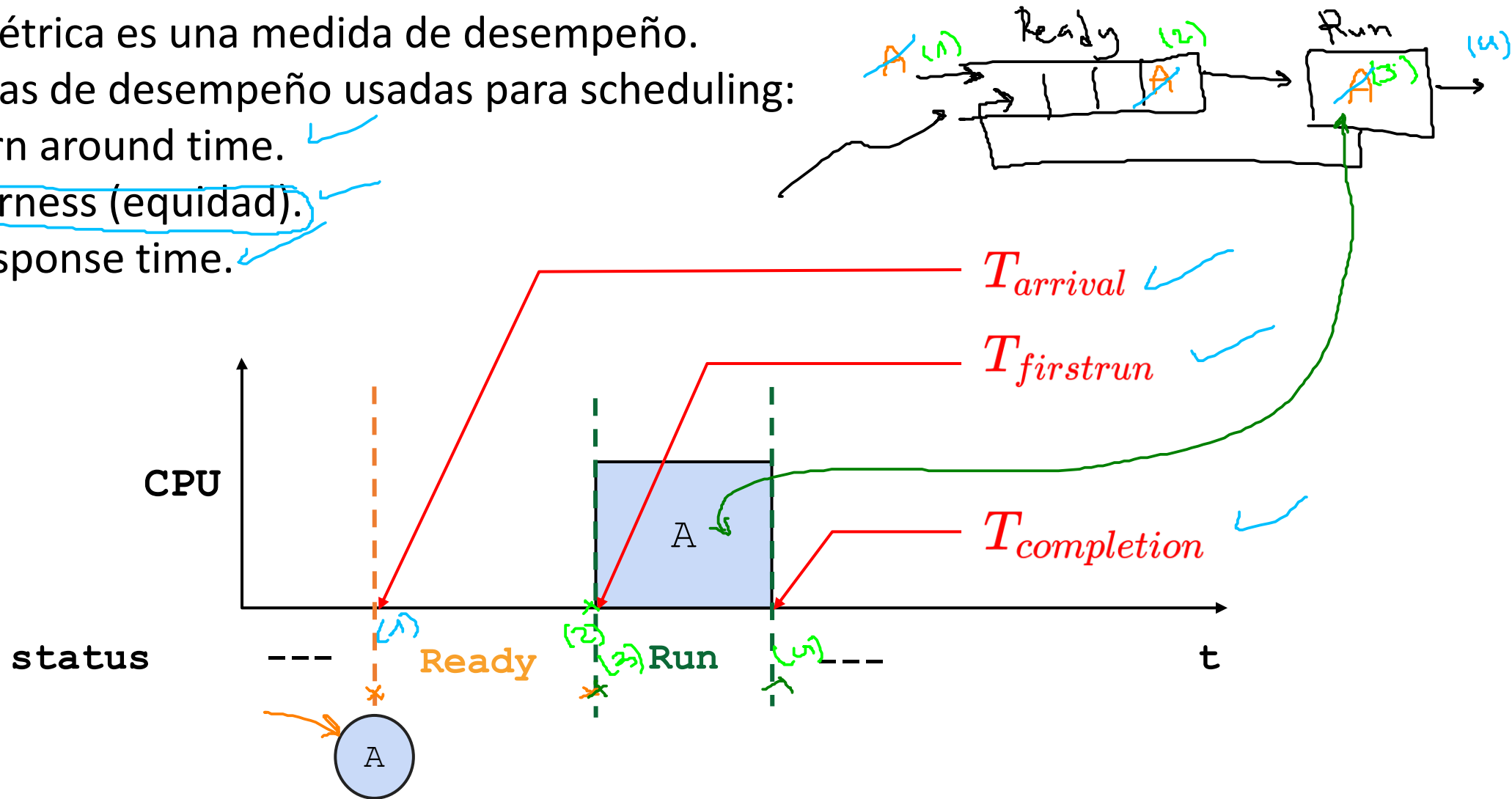
5. El tiempo de ejecución de cada trabajo es conocido (runtime).



Métricas de planificación

Métrica

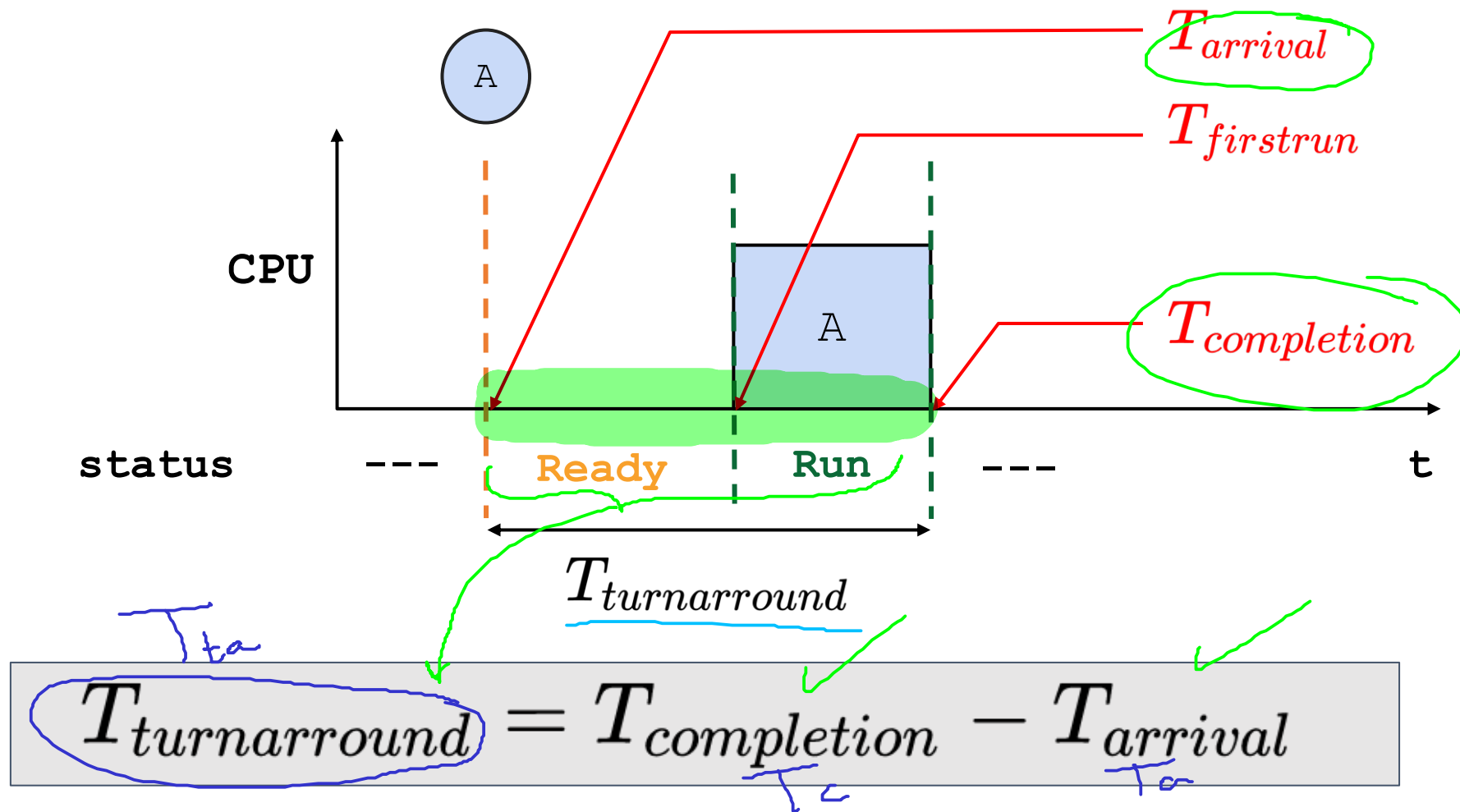
- Una métrica es una medida de desempeño.
- Métricas de desempeño usadas para scheduling:
 - Turn around time.
 - Fairness (equidad).
 - Response time.



Métricas de planificación

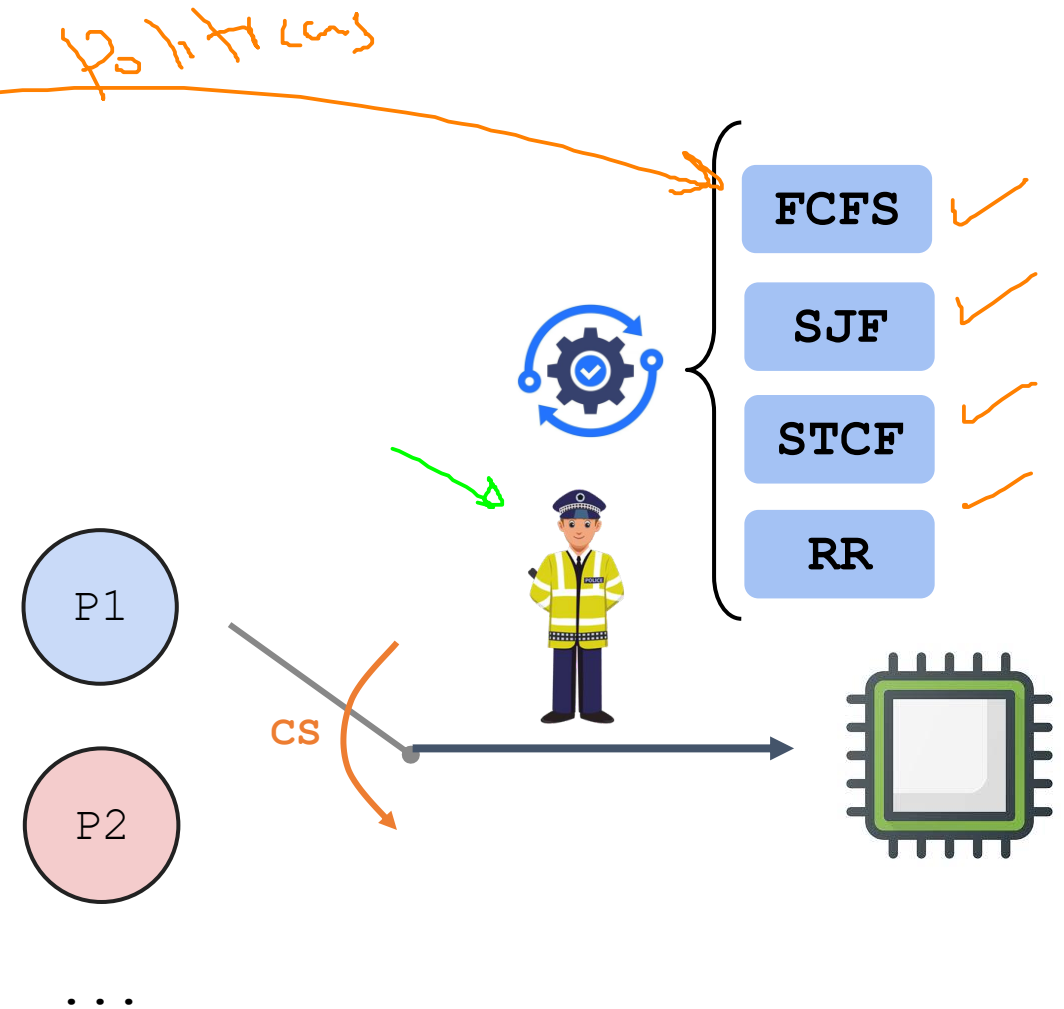
Turnaround Time

- Tiempo comprendido entre la llegada y la finalización de un proceso.



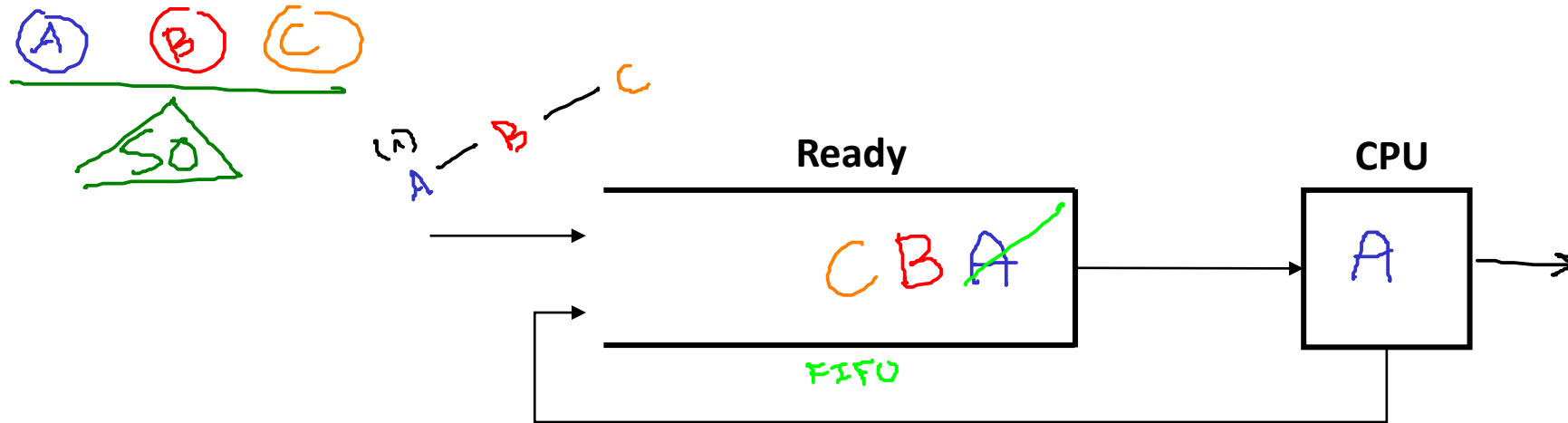
Algoritmos de planificación de procesos

- El planificador de procesos depende del algoritmo de planificación de procesos, el cual, basado en una política específica, asigna la CPU y mueve los diferentes trabajos a través del sistema.
- Algoritmos (no todos) de planificación de procesos
 - **First-Come, First-Served (FCFS).**
 - **Shortest Job First (SJF).**
 - **Shortest time to completion First (STCF).**
 - **Round robin (RR).**



First-Come, First-Served (FCFS)

- Los procesos son manejados de acuerdo al tiempo de llegada.
 - El primer trabajo que llega, es el primero en ser atendido.
- Simple y fácil de implementar
 - Cola **FifoFIFO (First-In, First-Out)**.
- **No apropiativo (Non preemptive)**: Solo se selecciona un nuevo proceso cuando el proceso anterior acaba.



First-Come, First-Served (FCFS)

- **Ejemplo:** Suponga que se tiene una situación en la que:
 - Tres procesos llegan a un sistema (A, B y C)
 - Llegan en el tiempo 0 en el orden: A - B - C ←
 - Cada proceso se ejecuta por 10 segundos.

¿Cual es el turnaround time promedio?

Proceso	Arrival time	Run-time
A	0	10
B	0	10
C	0	10

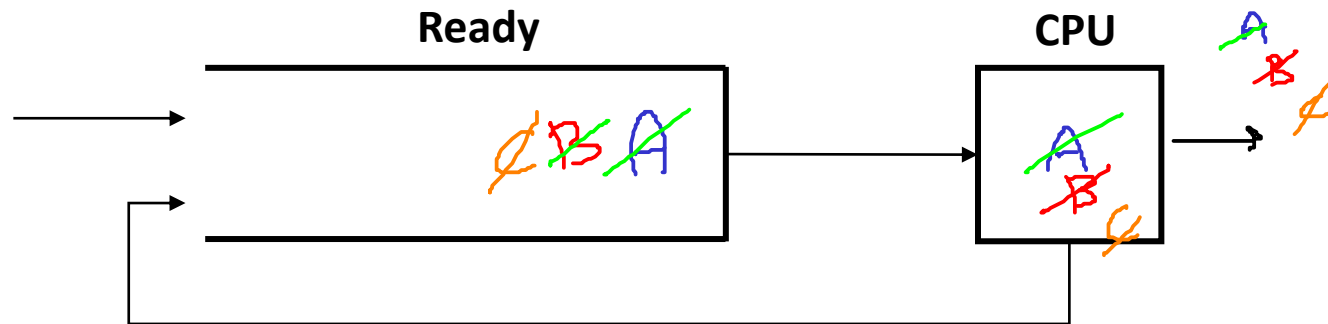
- Orden de llegada:

A - B - C
{^}



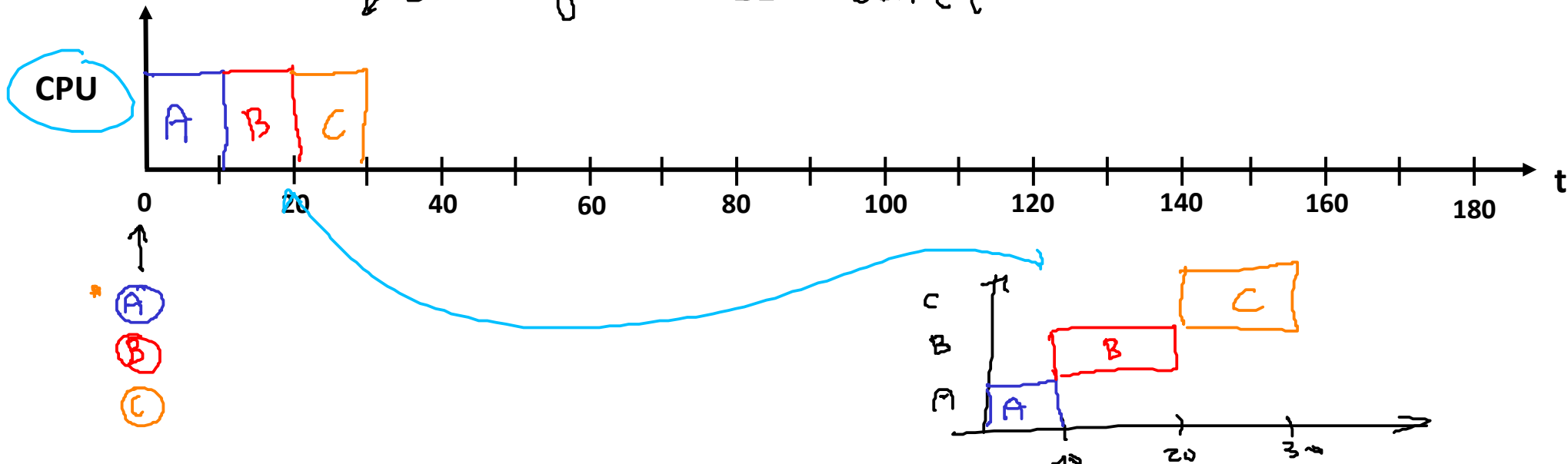
First-Come, First-Served (FCFS)

A — B — C
 ↘



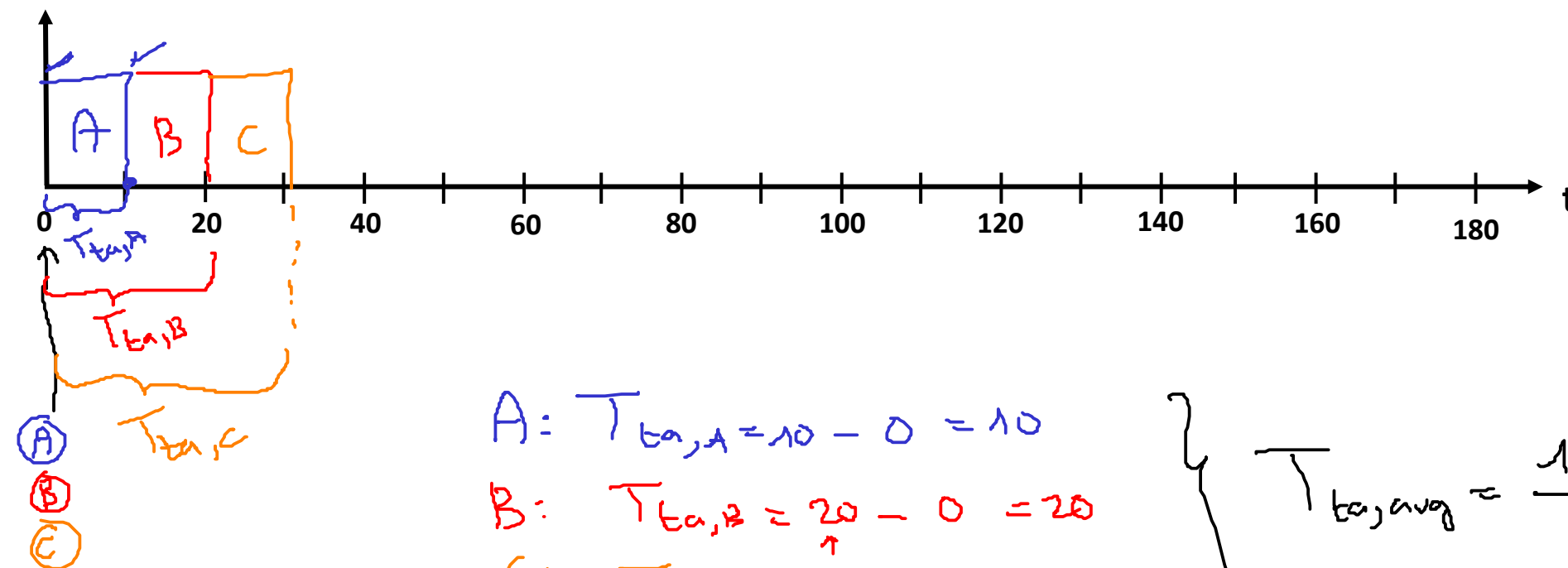
Proceso	Arrival time	Run-time
A	0	10 ✓
B	0	10 ✓
C	0	10 ✓

Diagrama de Gantt



First-Come, First-Served (FCFS)

Proceso	T_{ta}
A	10
B	20
C	30
avg	20



$$A: T_{ta,A} = 10 - 0 = 10$$

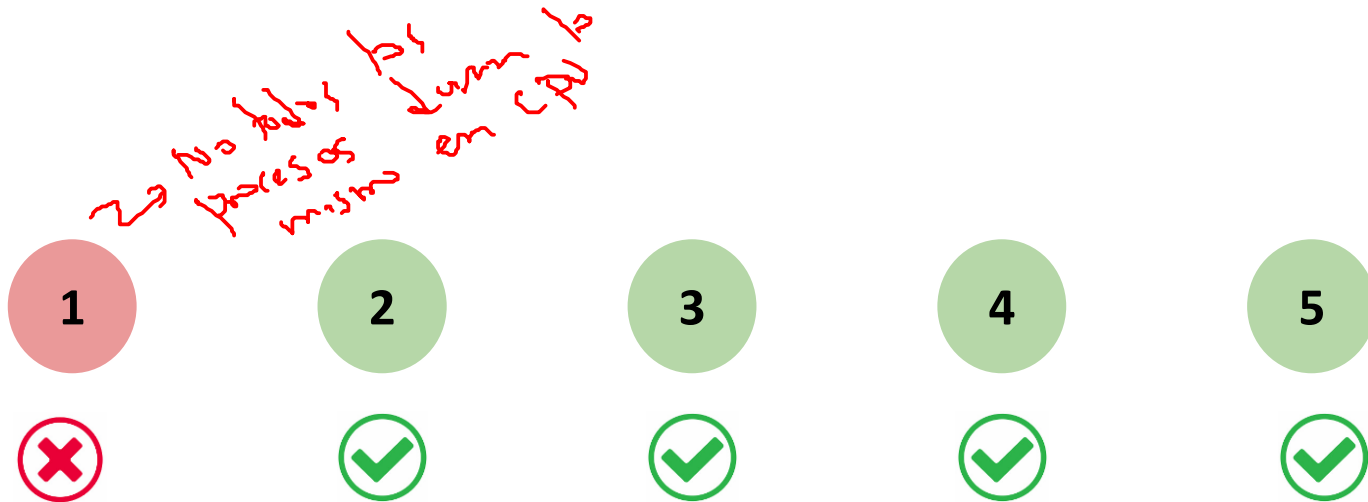
$$B: T_{ta,B} = 20 - 0 = 20$$

$$C: T_{ta,C} = 30 - 0 = 30$$

$$T_{ta,avg} = \frac{10 + 20 + 30}{3} = 20$$

Relajación de las suposiciones ideales

- ~~1. Cada trabajo se ejecuta por la misma cantidad de tiempo.~~
2. Todos los trabajos son iniciados al mismo tiempo (arrival time).
3. Una vez iniciado, cada trabajo se ejecuta hasta su finalización.
4. Todos los trabajos solo usan la CPU (no I/O).
5. El tiempo de ejecución de cada trabajo es conocido (runtime).



First-Come, First-Served (FCFS)

- **Ejemplo:** Suponga que se tiene una situación en la que:
 - Tres procesos llegan a un sistema (A, B y C)
 - Llegan en el tiempo 0 en el orden: A - B - C
 - **A se ejecuta por 100 seg**, B y C por 10 seg cada uno.

¿Cual es el turnarround time promedio?

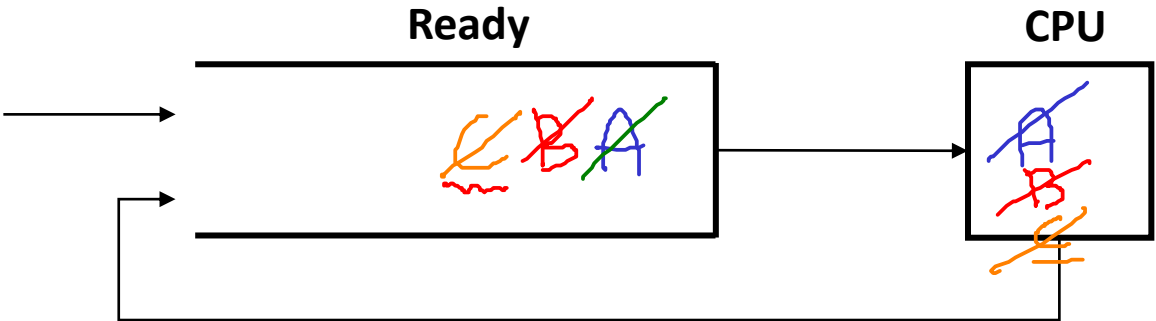
Proceso	Arrival time	Run-time
A	0	100
B	0	10
C	0	10

- Orden de llegada:

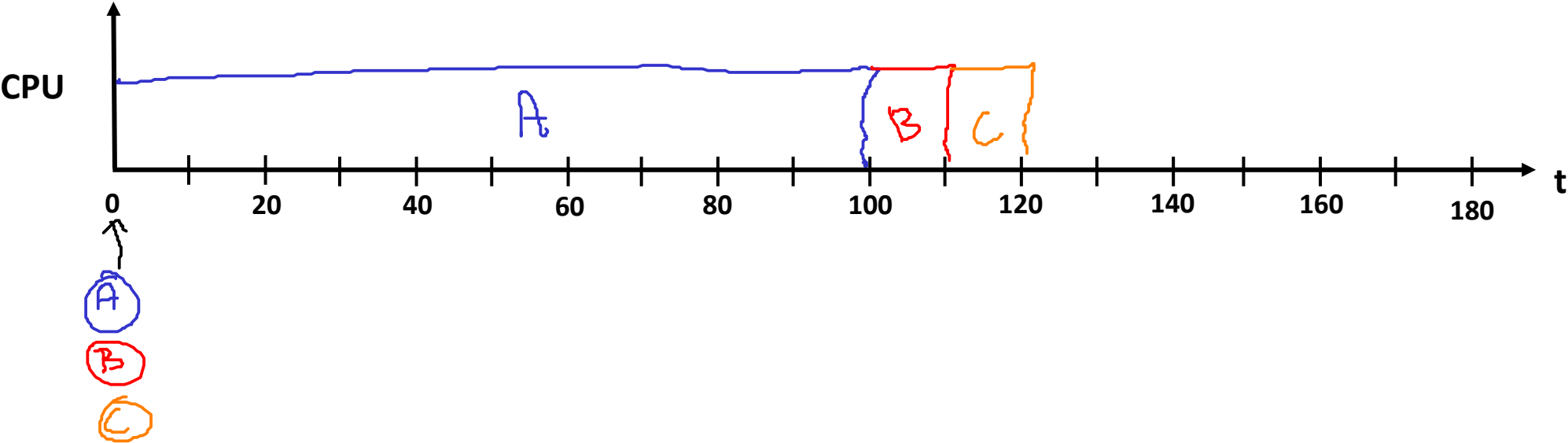
A - B - C

First-Come, First-Served (FCFS)

A (1) — B (2) — C (3)

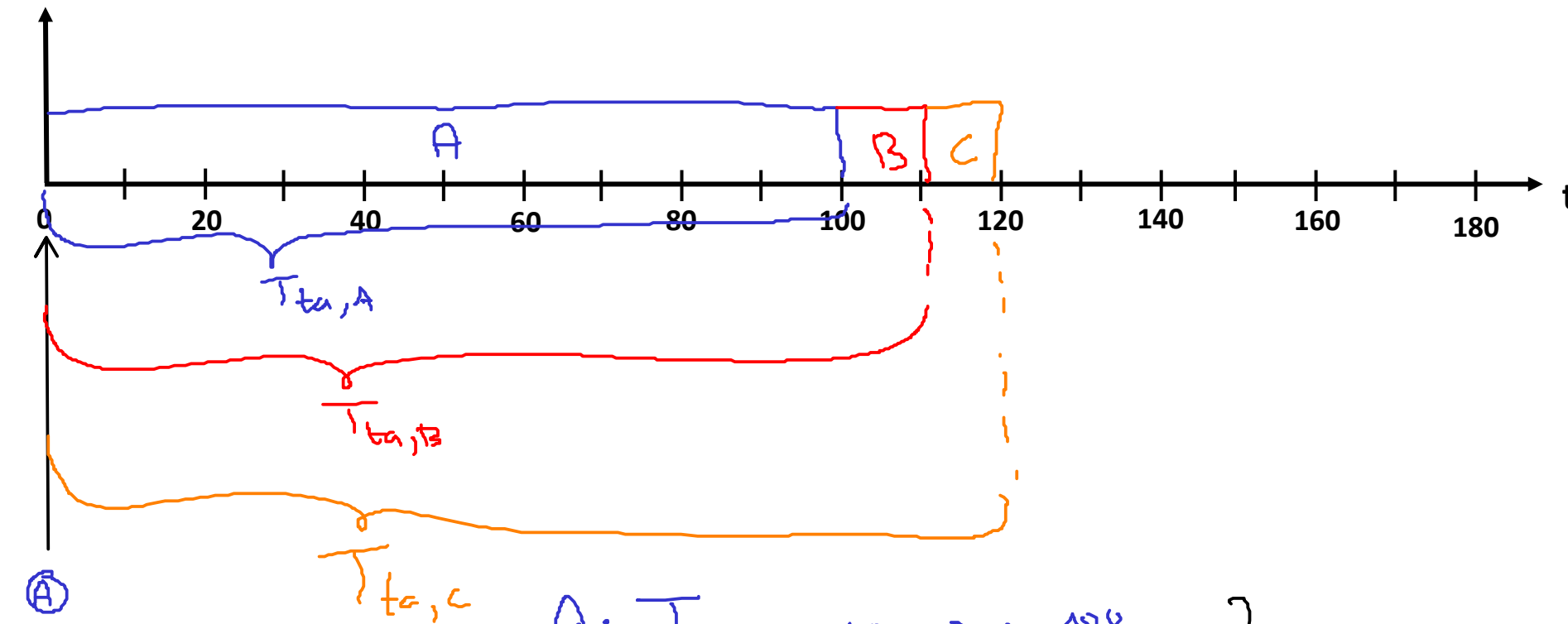


Proceso	Arrival time	Run-time
A	0 ✓	<u>100</u>
B	0 ✓	<u>10</u> ✓
C	0 ✓	10 ✓



First-Come, First-Served (FCFS)

Proceso	T_{ta}
A	100
B	110
C	120
avg	110



①
②
③

$$A: T_{ta,A} = 100 - 0 = 100$$

$$B: T_{ta,B} = 110 - 0 = 110$$

$$C: T_{ta,C} = 120 - 0 = 120$$

$$T_{ta,avg} = \frac{100 + 110 + 120}{3} = 110$$

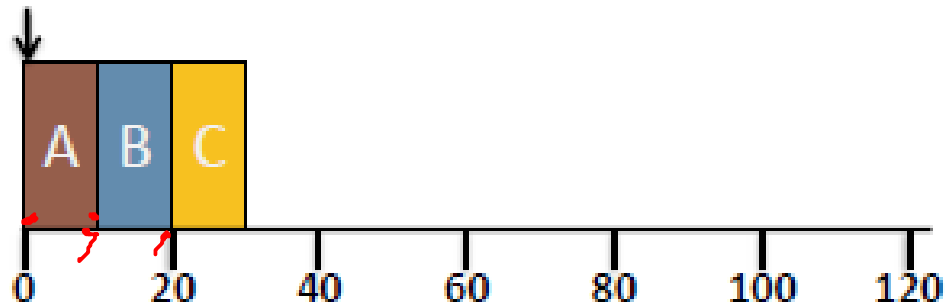
First-Come, First-Served (FCFS)

Comparación

Caso 1: FCFS



A(10), B(10), C(10)

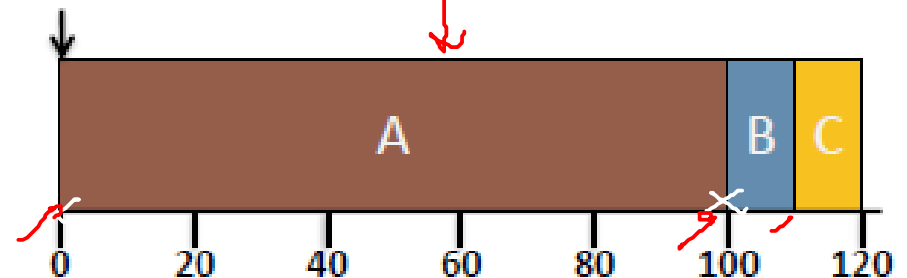


$$T_{\text{turnaround}} = (10 + 20 + 30)/3 = 20$$

Caso 2: FCFS



A(100), B(10), C(10)

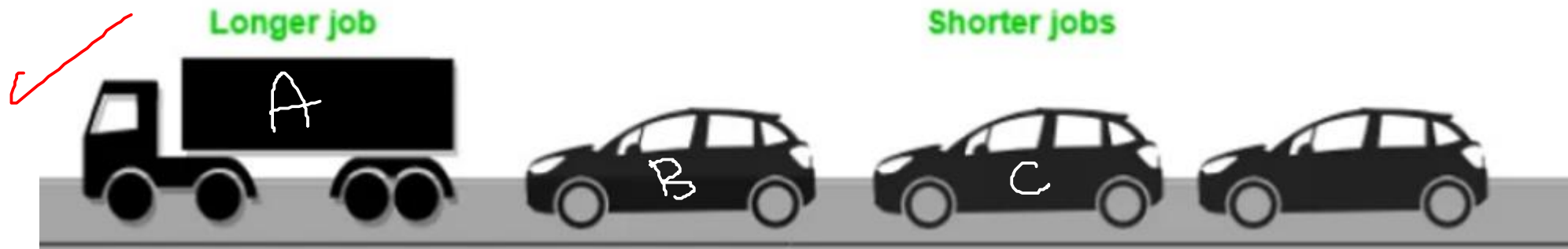


$$T_{\text{turnaround}} = (100 + 110 + 120)/3 = 110$$

Efecto Convoy

Efecto Convoy

- Se da cuando un proceso que requiere mucho tiempo de CPU va por delante (se ejecuta primero) de procesos ligeros. ✓
- Hace que los tiempos de respuesta (más adelante hablaremos de esta métrica) y finalización se resientan.
- El trabajo más grande es el que más influye en el tiempo de respuesta (turnaround time).



Shortest Job First (SJF)

- El tiempo de ejecución de cada tarea es variable (relajación de la suposición 1)
- La planificación de las tareas se hace de acuerdo a su tiempo de ejecución.
- El Scheduler (planificador) pone a ejecutar primero los procesos más cortos.
- **No apropiativo (Non preemptive)**: Solo se selecciona un nuevo proceso cuando el proceso anterior acaba.



Shortest Job First (SJF)

- **Ejemplo:** Suponga que se tiene una situación en la que:
 - Tres procesos llegan a un sistema (A, B y C)
 - Llegan en el tiempo 0 en el orden: A - B - C
 - A se ejecuta por 100 seg, B y C por 10 seg cada uno.

¿Cual es el turnaround time promedio?

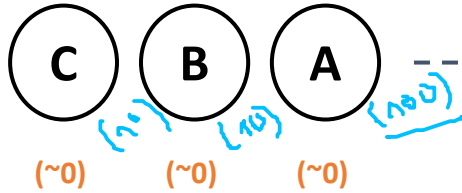
Proceso	Arrival time	Run-time
A	0	100
B	0	10
C	0	10

- Orden de llegada:
A - B - C



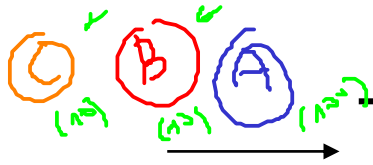
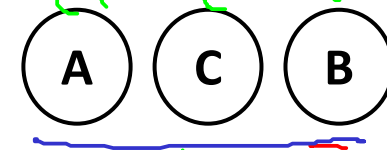
Shortest Job First (SJF)

Orden de llegada



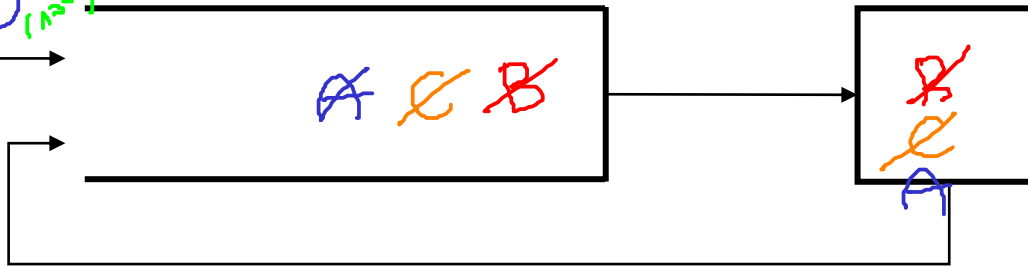
Reordenamiento

Orden de ejecución



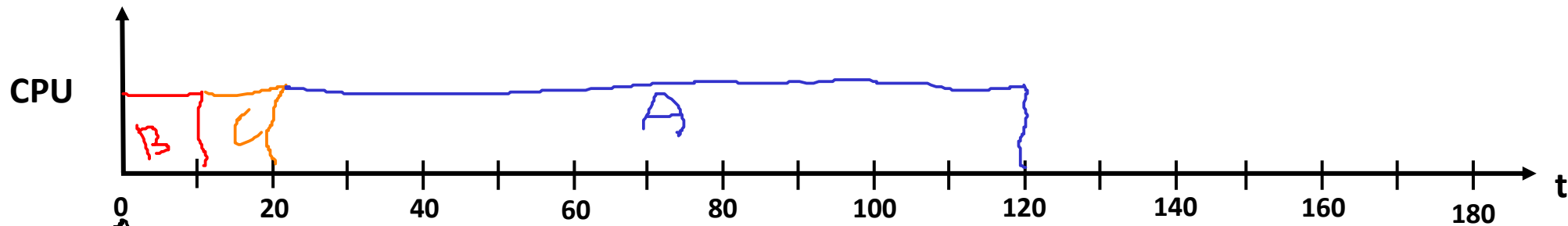
Ready

CPU



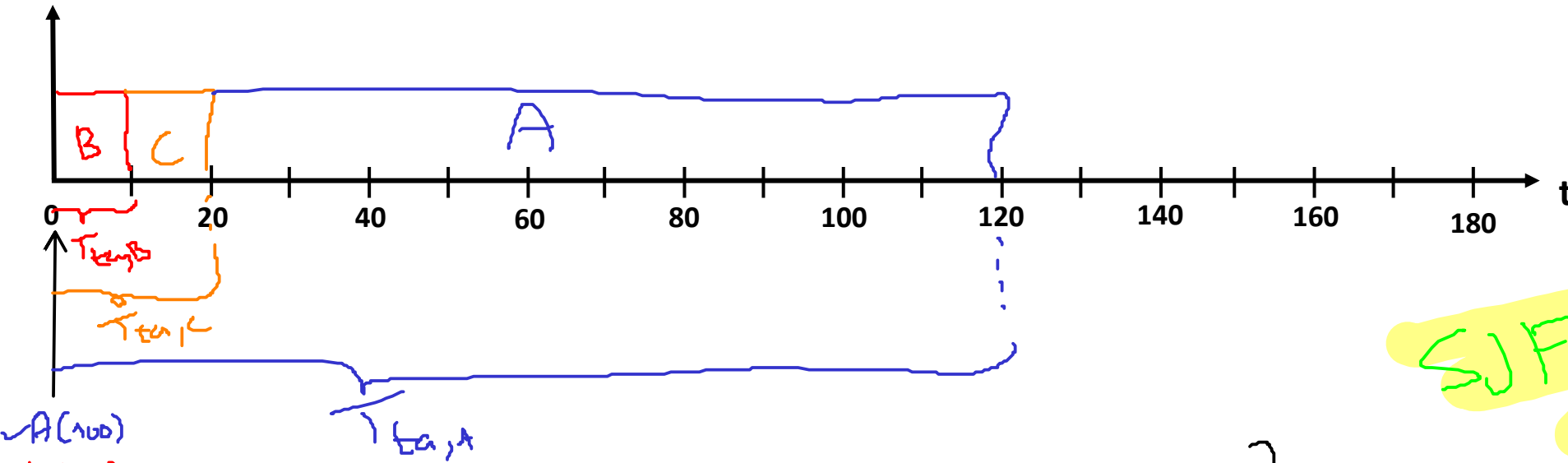
Proceso	Arrival time	Run-time
A	0	<u>100</u>
B	0	<u>10</u>
C	0	<u>10</u>

SJF



Shortest Job First (SJF)

Proceso	T_{ta}
A	120
B	10
C	20
avg	50



$\checkmark A(100)$
 $\checkmark B(10)$
 $\checkmark C(10)$

$$T_{ta,A} = 120 - 0 = 120$$

$$T_{ta,B} = 10 - 0 = 10$$

$$T_{ta,C} = 20 - 0 = 20$$

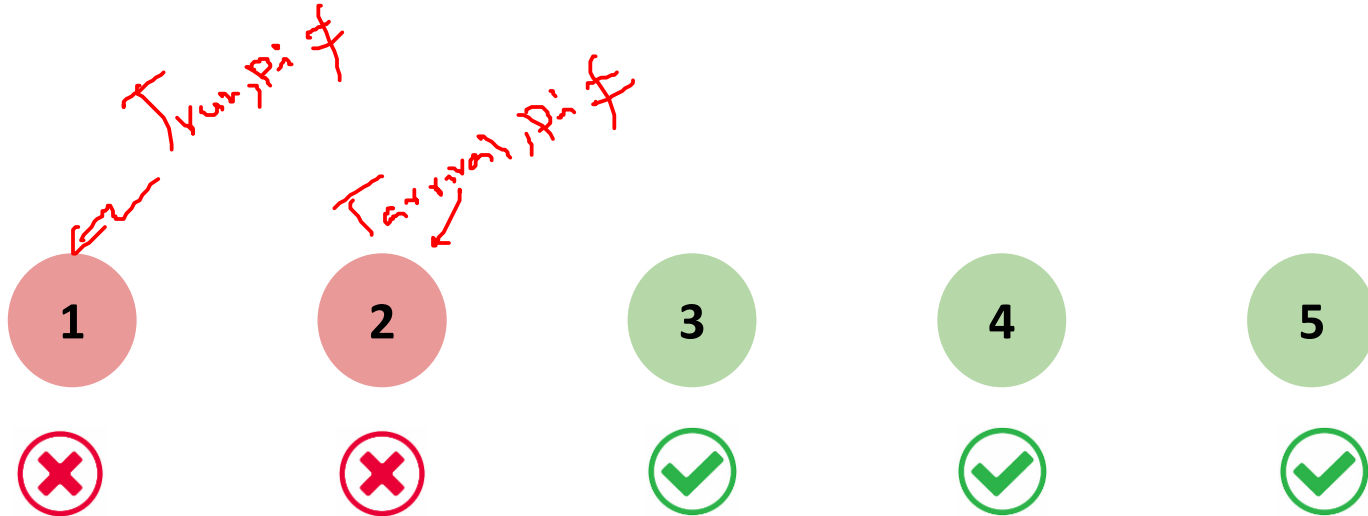
$$T_{ta,avg} = \frac{120 + 10 + 20}{3} = 50$$

SJF mejor que FIFO

$$T_{ta, SJF (avg)} < T_{ta, FIFO (avg)}$$

Relajación de las suposiciones ideales

- ~~1. Cada trabajo se ejecuta por la misma cantidad de tiempo.~~
- ~~2. Todos los trabajos son iniciados al mismo tiempo (arrival time).~~ ✓
3. Una vez iniciado, cada trabajo se ejecuta hasta su finalización.
4. Todos los trabajos solo usan la CPU (no I/O).
5. El tiempo de ejecución de cada trabajo es conocido (runtime).



Shortest Job First (SJF)

- **Ejemplo:** Suponga que se tiene una situación en la que:
 - Tres procesos llegan a un sistema (A, B y C)
 - A llega en $t = 0$ y B y C llegan en $t = 20$.
 - A se ejecuta por 100 seg, B y C por 10 seg cada uno.

¿Cual es el turnarround time promedio?

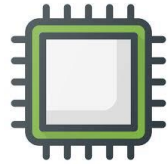
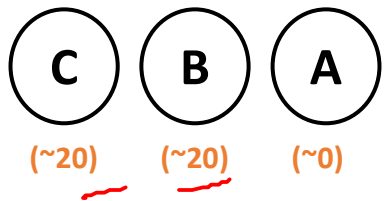
Proceso	Arrival time	Run-time
A	0	100
B	20	10
C	20	10

Handwritten notes in red ink:

- A circle containing the number 2, with an arrow pointing to the 'Arrival time' column.
- A circle containing the number 1, with an arrow pointing to the 'Run-time' column.
- The text "B y C llegan de A" (B and C arrive from A) written diagonally.

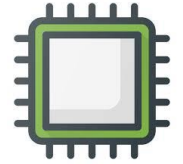
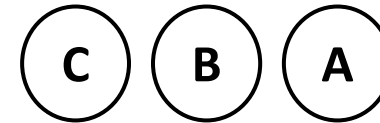
Shortest Job First (SJF)

Orden de llegada

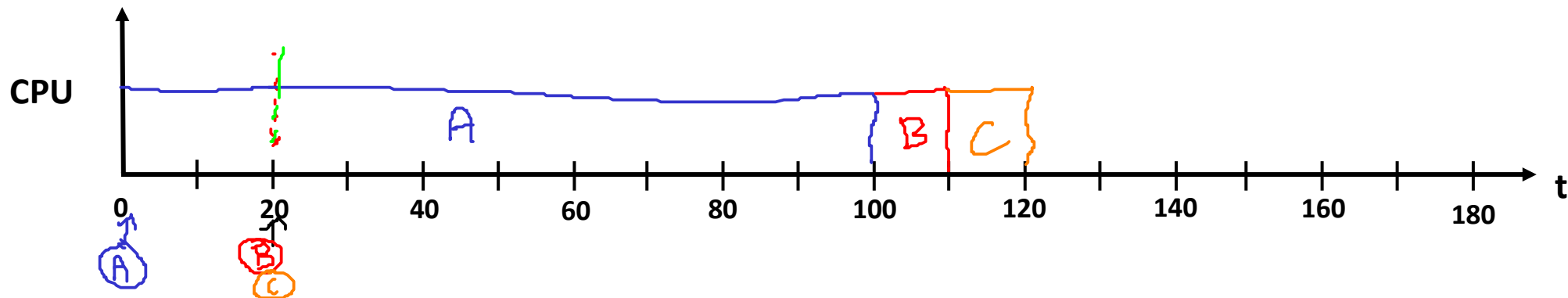


Reordenamiento

Orden de ejecución

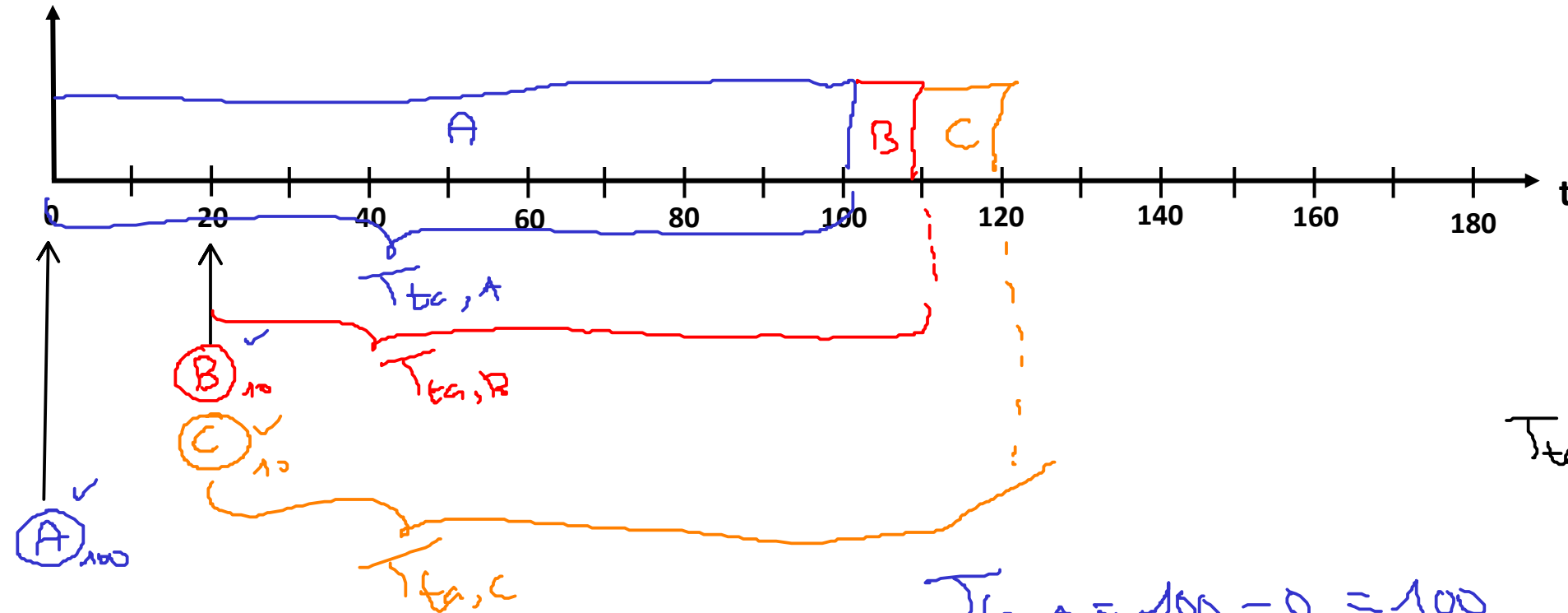


Proceso	Arrival time	Run-time
A	0	100
B	20	10
C	20	10



Shortest Job First (SJF)

Proceso	T_{ta}
A	100
B	90
C	100
avg	96,67



$$T_{ta,A} = 100 - 0 = 100$$

$$T_{ta,B} = 110 - 20 = 90$$

$$T_{ta,C} = 120 - 20 = 100$$

$$T_{ta,avg} = \frac{100 + 90 + 100}{3}$$

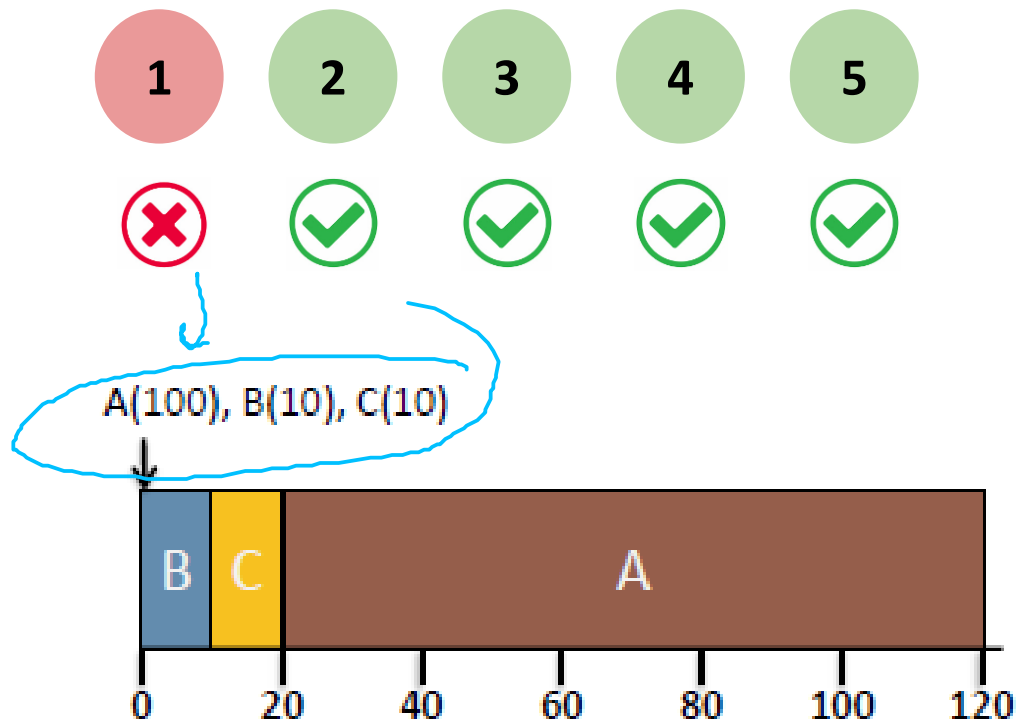
$$= \frac{290}{3} = 96,67$$



Shortest Job First (SJF)

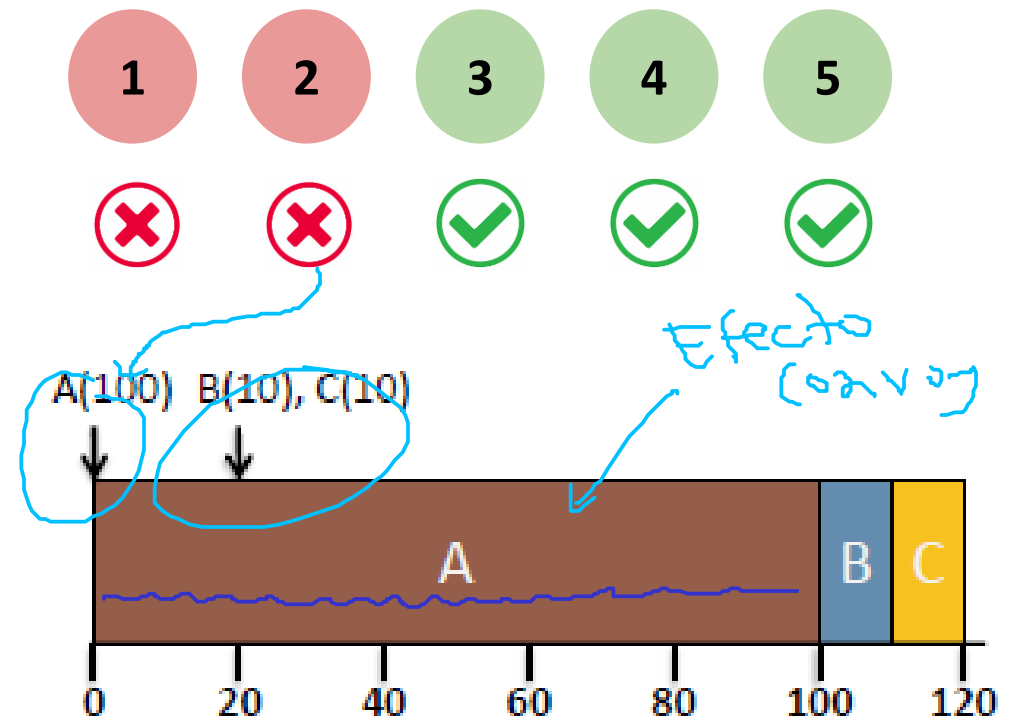
Comparación

Caso 1: SJF



$$T_{turnaround} = (10 + 20 + 120)/3 = 50$$

Caso 2: SJF



$$T_{turnaround} = (100 + 90 + 100)/3 = 96.7$$

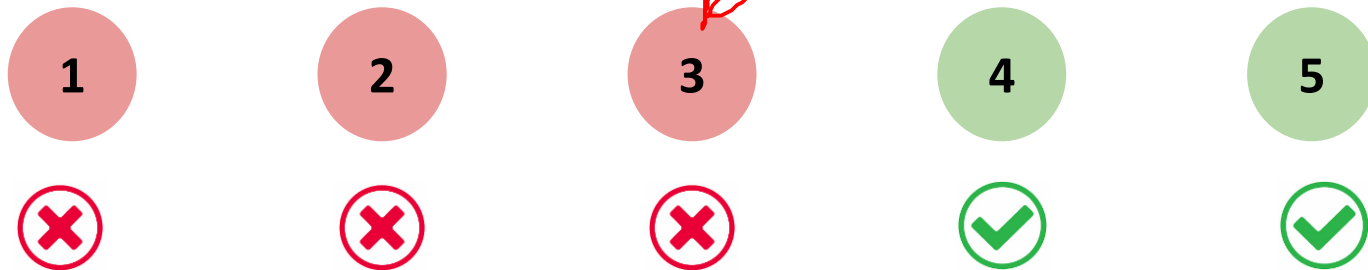
No basta solo con ordenar

Shortest Time-to-Completion First (STCF)

- Versión apropiativa (preemptive) del algoritmo SJF. *La CPU se le puede quitar al proceso que la está usando.*
- El procesador se asigna al trabajo que está más próximo a completarse.
- Un trabajo en ejecución puede expropiarse, si hay un trabajo nuevo (en la cola READY) con un menor tiempo de ejecución.
- **Funcionamiento básico:** Si un nuevo trabajo entra al sistema:
 - Compare, incluyendo los trabajos restantes y el nuevo trabajo.
 - Asigne la CPU al trabajo que requiera el menor tiempo para terminar.

Relajando las suposiciones ideales

- ~~1. Cada trabajo se ejecuta por la misma cantidad de tiempo.~~
- ~~2. Todos los trabajos son iniciados al mismo tiempo (arrival time).~~
- ~~3. Una vez iniciado, cada trabajo se ejecuta hasta su finalización.~~
4. Todos los trabajos solo usan la CPU (no I/O).
5. El tiempo de ejecución de cada trabajo es conocido (runtime).



Shortest Time-to-Completion First (STCF)

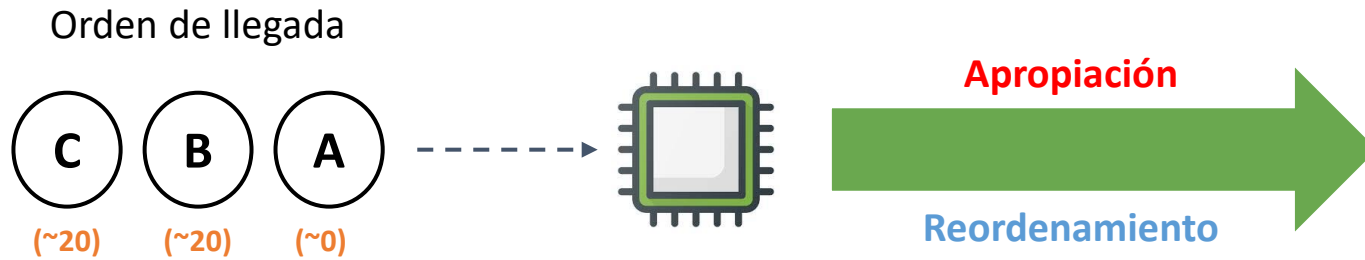
- **Ejemplo:** Suponga que se tiene una situación en la que:
 - Tres procesos llegan a un sistema (A, B y C)
 - A llega en $t = 0$ y B y C llegan en $t = 20$.
 - A se ejecuta por 100 seg, B y C por 10 seg cada uno.

¿Cual es el turnaround time promedio?

Proceso	Arrival time	Run-time
A	0	100
B	20	10
C	20	10



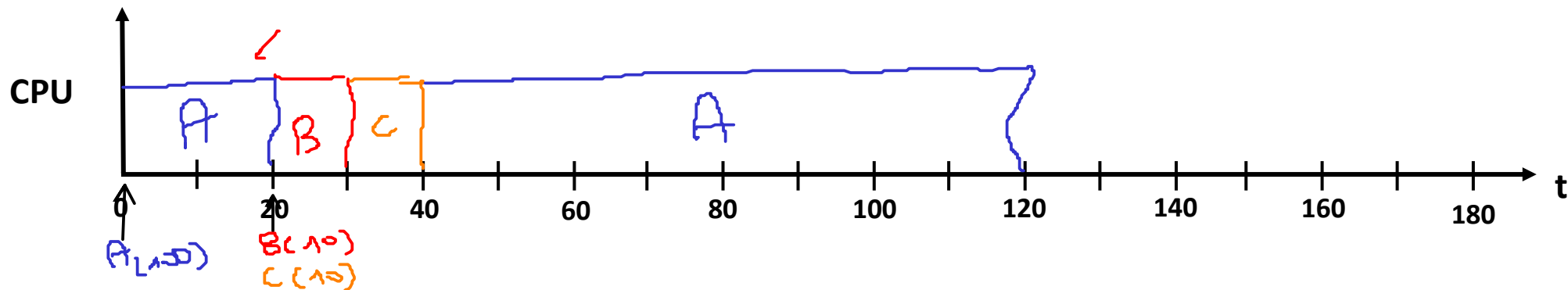
Shortest Time-to-Completion First (STCF)



Proceso	Arrival time	Run-time
A	0	100
B	20	10
C	20	10

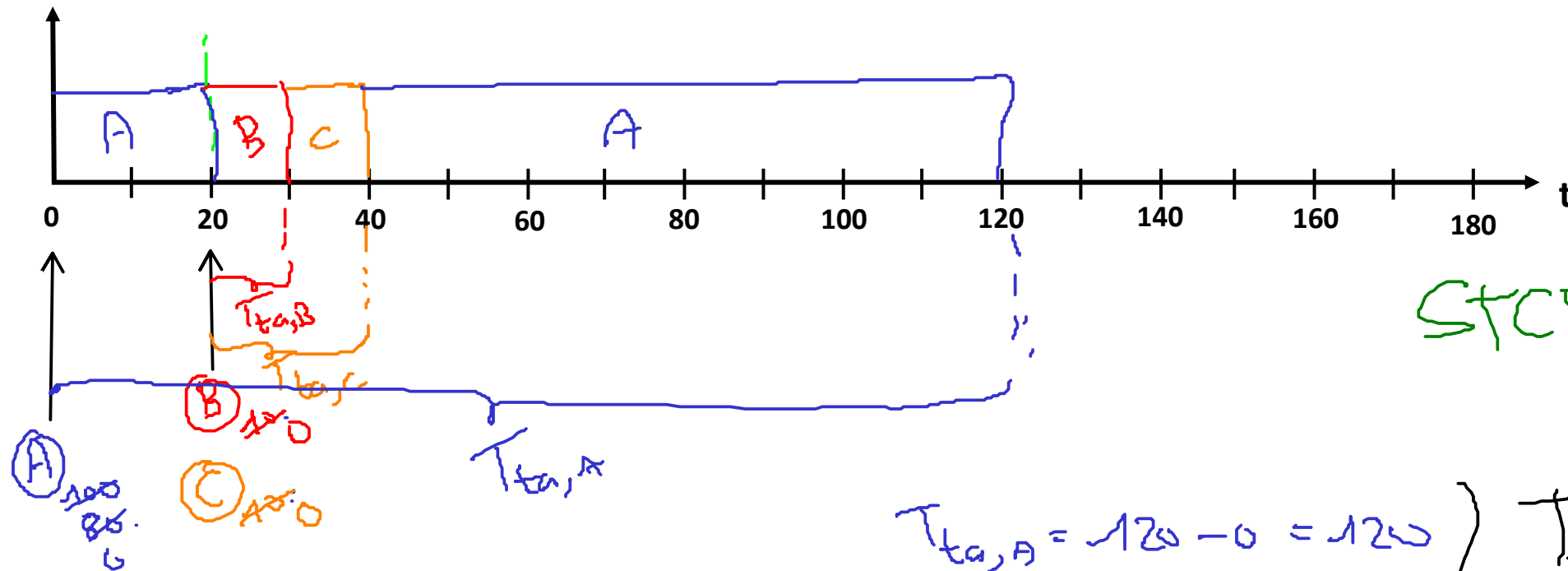


~~A = 100~~ ~~80~~ 10
~~B = 10~~ 0
~~C = 10~~ 0



Shortest Time-to-Completion First (STCF)

Proceso	T_{ta}
A	120
B	10
C	20
avg	50



STCF es mejor que el SJF

$$T_{ta,A} = 120 - 0 = 120$$

$$T_{ta,B} = 30 - 20 = 10$$

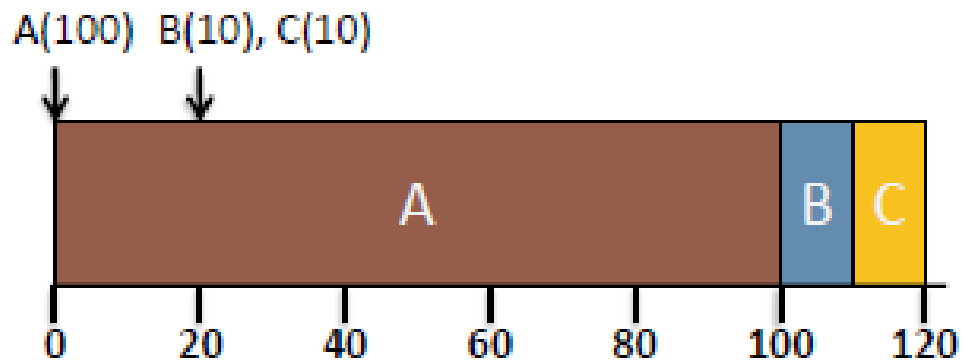
$$T_{ta,C} = 40 - 20 = 20$$

$$T_{ta,avg} = \frac{120 + 10 + 20}{3} = 50$$

Shortest Time-to-Completion First (STCF)

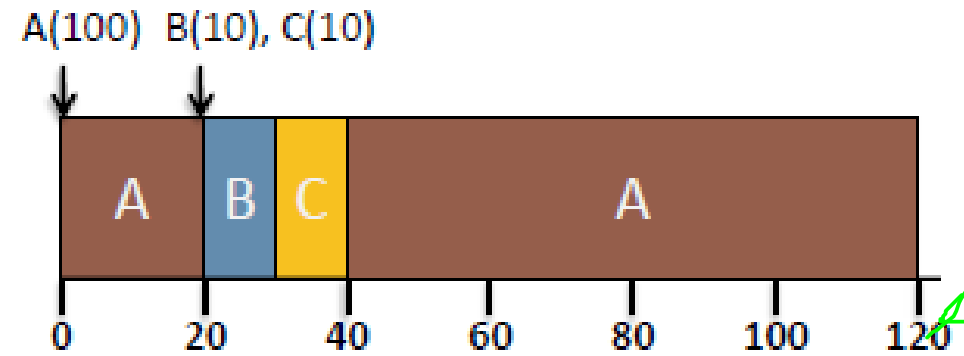
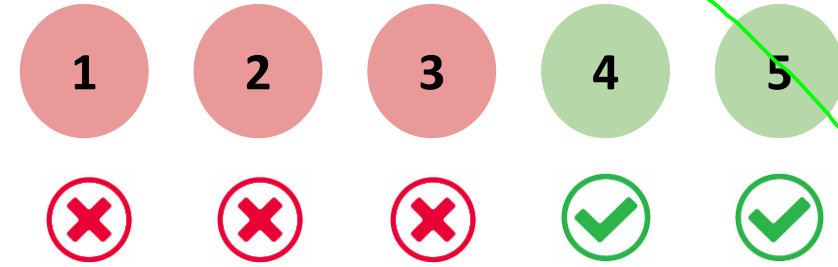
Comparación

Caso 2: SJF



$$T_{\text{turnaround}} = (100 + 90 + 100)/3 = 96.7$$

Caso 3: STCF



$$T_{\text{turnaround}} = (120 + 10 + 20)/3 = 50$$

Resumen algoritmos de planificación (FCFS, SJF, STCF)

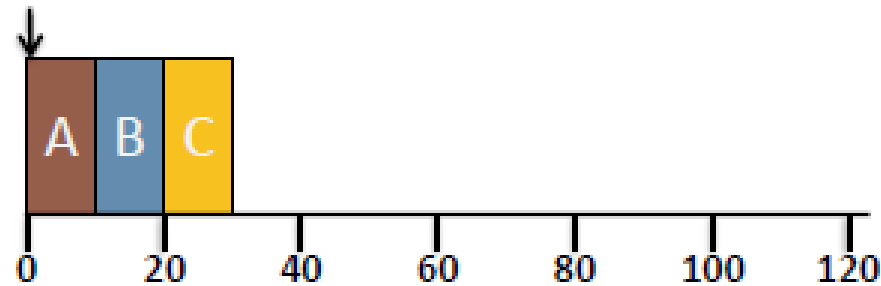
- **Ejemplo 1:** Suponga que se tiene una situación en la que:

- Tres procesos llegan a un sistema (A, B y C)
- Llegan en el tiempo 0 en el orden: A - B - C
- Cada proceso se ejecuta por 10 segundos.

Proceso	Arrival time	Run-time
A	0	10
B	0	10
C	0	10

FCFS (FIFO)

A(10), B(10), C(10)



$$T_{turnaround} = (10 + 20 + 30) / 3 = 20$$



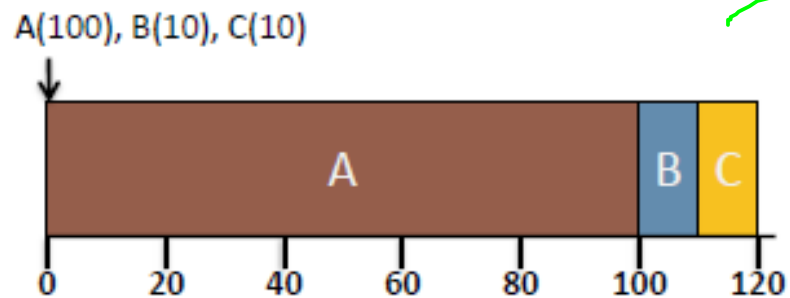
Resumen algoritmos de planificación (FCFS, SJF, STCF)

- **Ejemplo 2:** Suponga que se tiene una situación en la que:

- Tres procesos llegan a un sistema (A, B y C)
- Llegan en el tiempo 0 en el orden: A - B - C
- **A se ejecuta por 100 seg**, B y C por 10 seg cada uno.

Proceso	Arrival time	Run-time
A	0	100
B	0	10
C	0	10

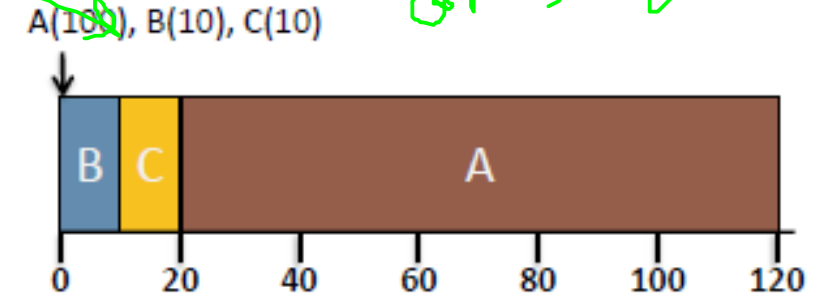
FCFS (FIFO)



$$T_{turnaround} = (100 + 110 + 120)/3 = 110$$

SJF

Effect =
turn =



$$T_{turnaround} = (10 + 20 + 120)/3 = 50$$

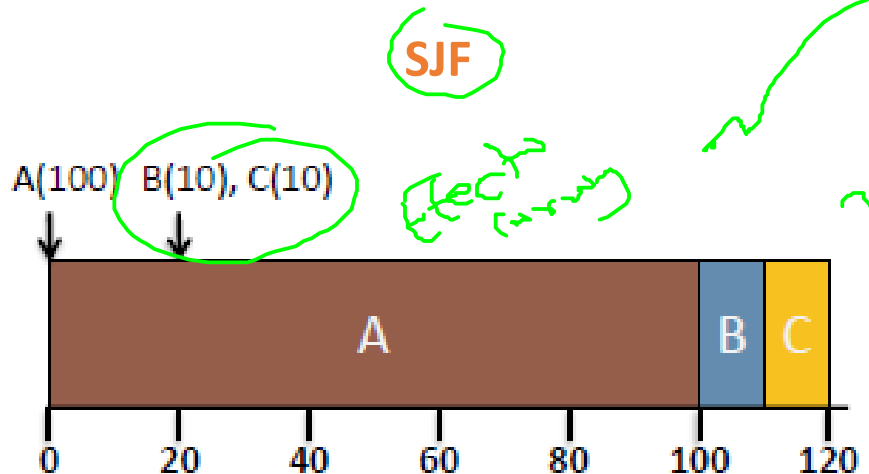
Resumen algoritmos de planificación (FCFS, SJF, STCF)

- **Ejemplo 3:** Suponga que se tiene una situación en la que:

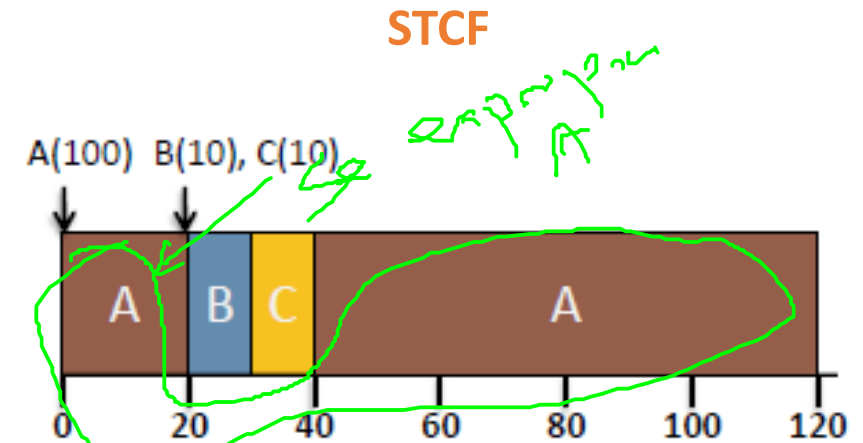
- Tres procesos llegan a un sistema (A, B y C)
- A llega en $t = 0$ y B y C llegan en $t = 20$.
- A se ejecuta por 100 seg, B y C por 10 seg cada uno.

mejor

Proceso	Arrival time	Run-time
A	0	100
B	20	10
C	20	10



$$T_{\text{turnaround}} = (100 + 90 + 100)/3 = \underline{96.7}$$

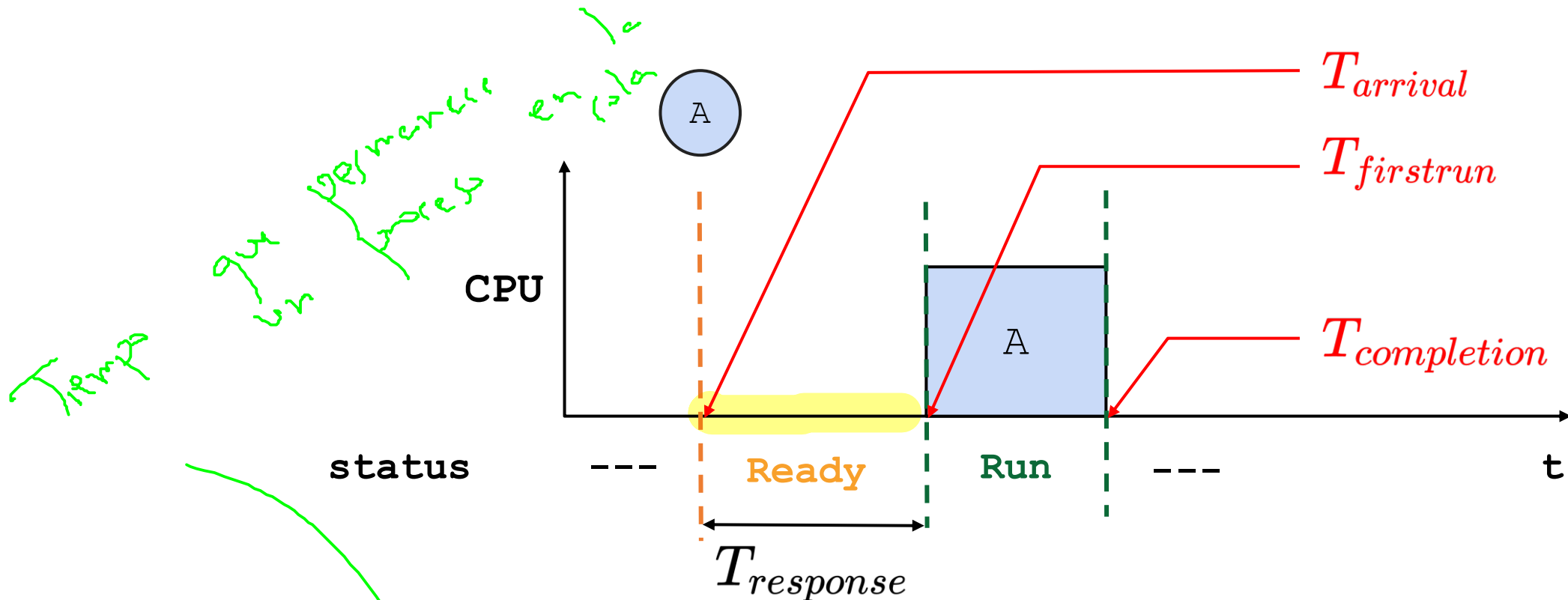


$$T_{\text{turnaround}} = (120 + 10 + 20)/3 = \underline{50}$$



Response time

- Tiempo medido desde que el trabajo llega hasta que es programado por primera vez.



$$T_{response} = T_{firstrun} - T_{arrival}$$

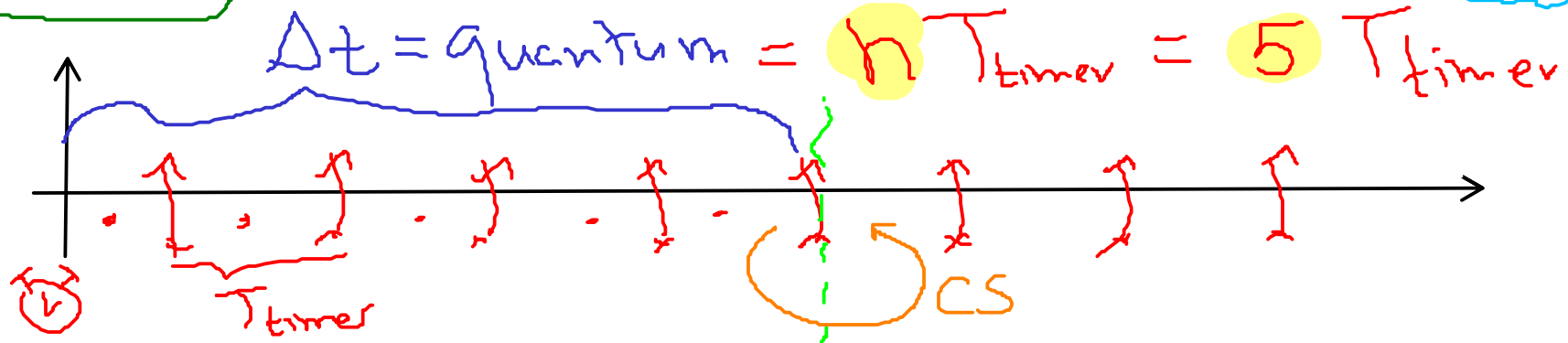


Round Robin (RR)

Round Robin (RR)

Planificación por porciones de tiempo (**time slicing**)

- Ejecute un trabajo por una **porción de tiempo**, luego ejecute el siguiente trabajo en la cola de **procesos listos**
 - La porción de tiempo es llamada **quantum**
- Continúe haciendo lo mismo hasta que todos los **trabajos finalicen**.
- La duración de la porción de tiempo debe ser un **múltiplo** del periodo de **interrupción del timer**.
- RR es **justo (fair)**, pero tiene un **bajo desempeño** en terminos de turnaround time.



Round Robin (RR)

Ejemplo:

Suponga que se tiene una situación en la que:

- Tres procesos llegan a un sistema (A, B y C)
- Llegan en el tiempo 0 en el orden: A - B - C
- El tiempo de ejecución de cada uno de los procesos es de 30 seg.

Proceso	Arrival time	Run-time
A	0	30
B	0	30
C	0	30

Hallar:

- Turnaround time

$$T_{turnaround} = T_{completion} - T_{arrival}$$



- Response time

$$T_{response} = T_{firstrun} - T_{arrival}$$



Round Robin (RR)

05/03/2026

Avisos: 1. Hoy se activa el Quiz
Semana 3 (scheduling)
2. Curso de RHA (Enviar
aviso)

Ejemplo:

Suponga que se tiene una situación en la que:

- Tres procesos llegan a un sistema (A, B y C) ✓
- Llegan en el tiempo 0 en el orden: A - B - C ✓
- El tiempo de ejecución de cada uno de los procesos es de 30 seg. ✓

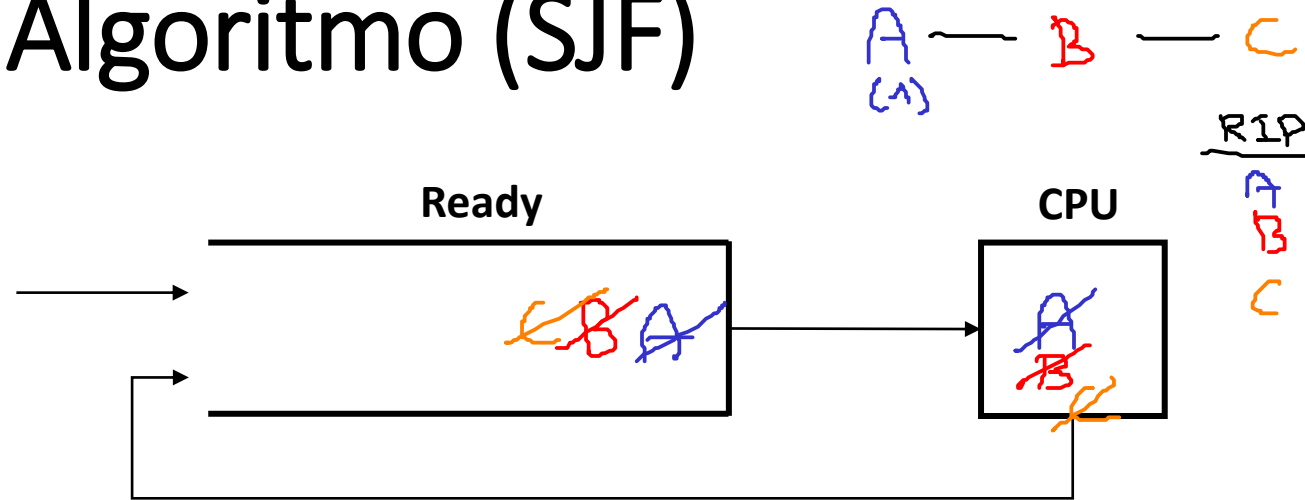
Proceso	Arrival time	Run-time
A	0	30
B	0	30
C	0	30

Calcule el turnaround time y el response time empleando el **Algoritmo SJF**.

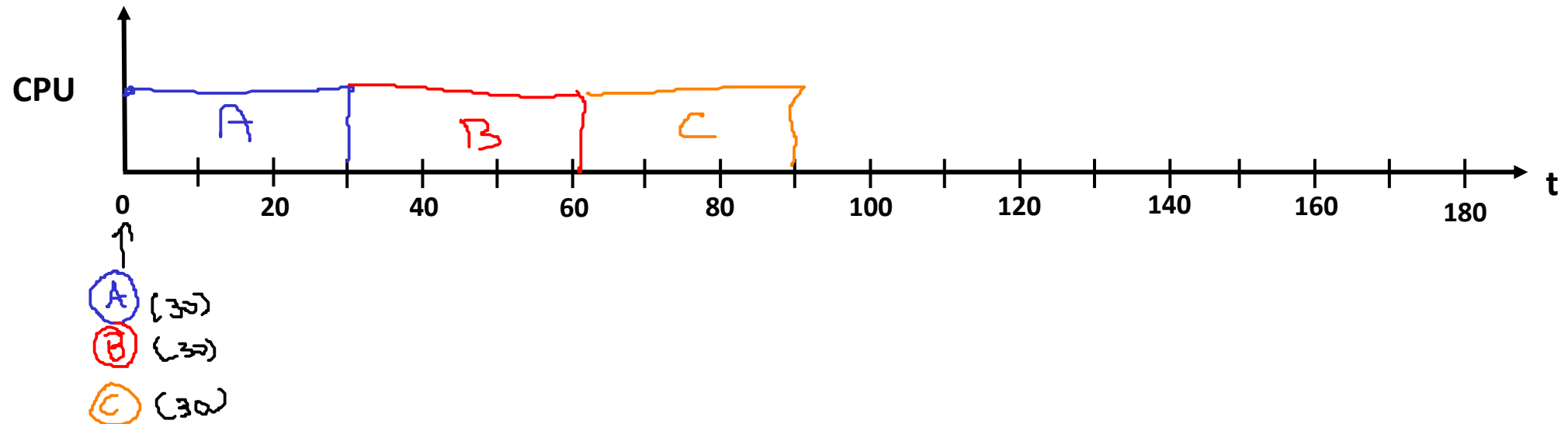
El mas corto primero



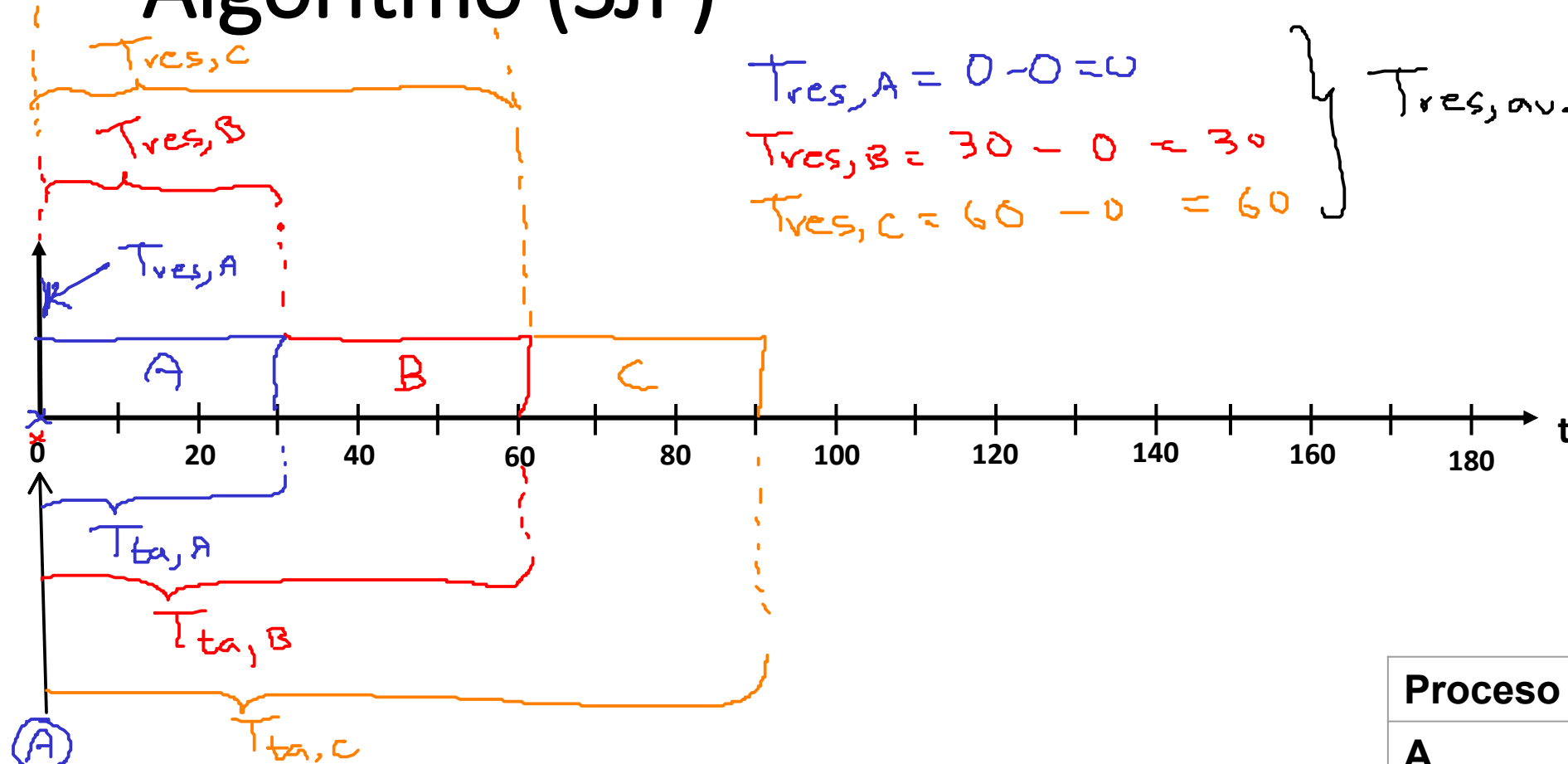
Algoritmo (SJF)



Proceso	Arrival time	Run-time
A	0	30
B	0	30
C	0	30



Algoritmo (SJF)



$$T_{res,A} = 0 - 0 = 0$$

$$T_{res,B} = 30 - 0 = 30$$

$$T_{res,C} = 60 - 0 = 60$$

$$T_{res,avg} = \frac{0 + 30 + 60}{3} = 30$$

$$T_{ta,A} = 30 - 0 = 30$$

$$T_{ta,B} = 60 - 0 = 60$$

$$T_{ta,C} = 90 - 0 = 90$$

$$T_{ta,avg} = \frac{180}{3} = 60$$

Proceso	T_{ta}	T_{res}
A	30	0
B	60	30
C	90	60
avg	60	30

Round Robin (RR)

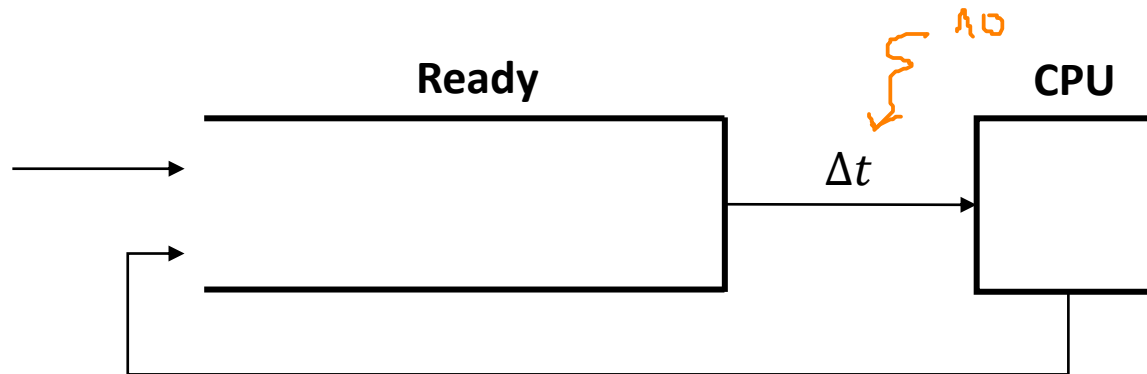
Ejemplo:

Suponga que se tiene una situación en la que:

- Tres procesos llegan a un sistema (A, B y C)
- Llegan en el tiempo 0 en el orden: A - B - C
- El tiempo de ejecución de cada uno de los procesos es de 30 seg.

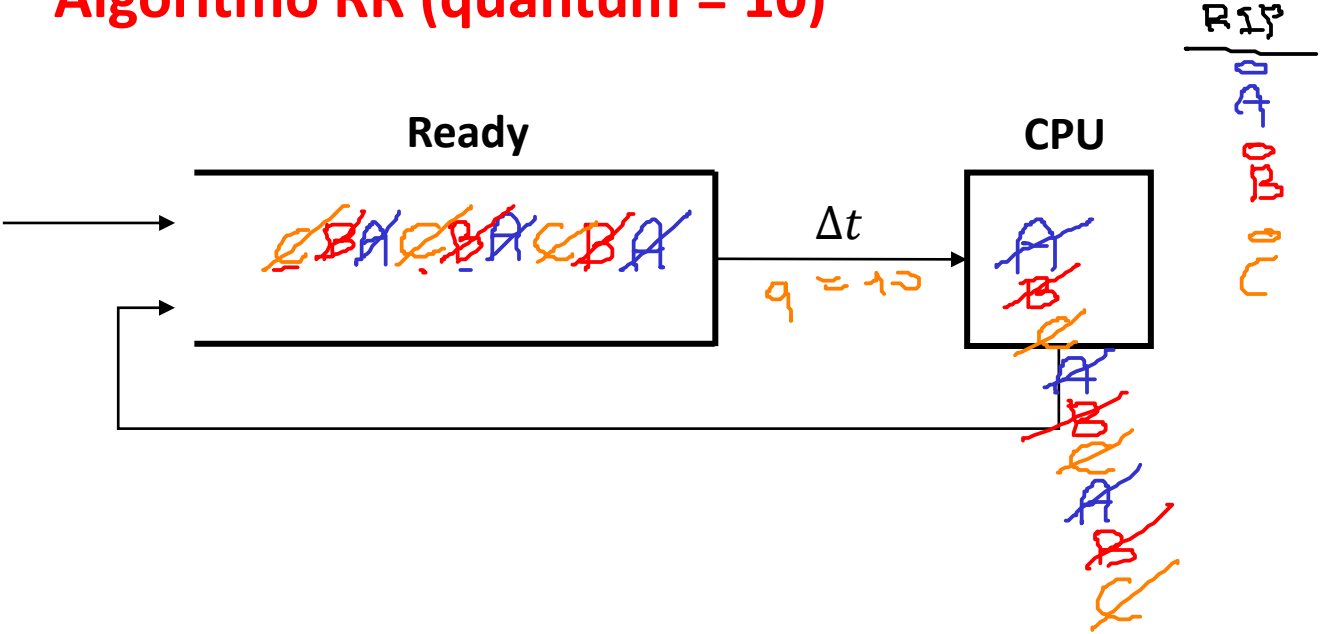
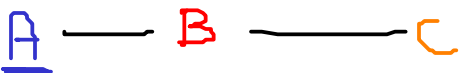
Proceso	Arrival time	Run-time
A	0	30
B	0	30
C	0	30

Calcule el turnaround time y el response time empleando el **Algoritmo RR** con un quantum de 10 seg.

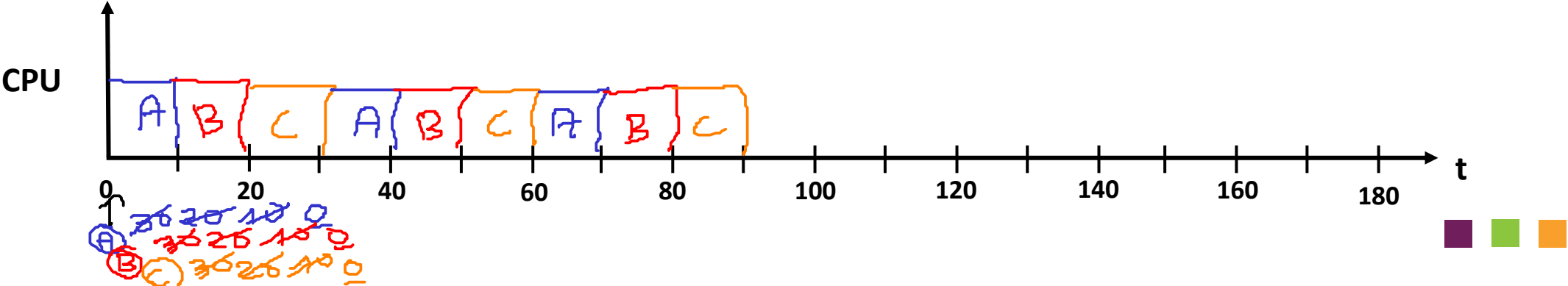


Round Robin (RR)

Algoritmo RR (quantum = 10)

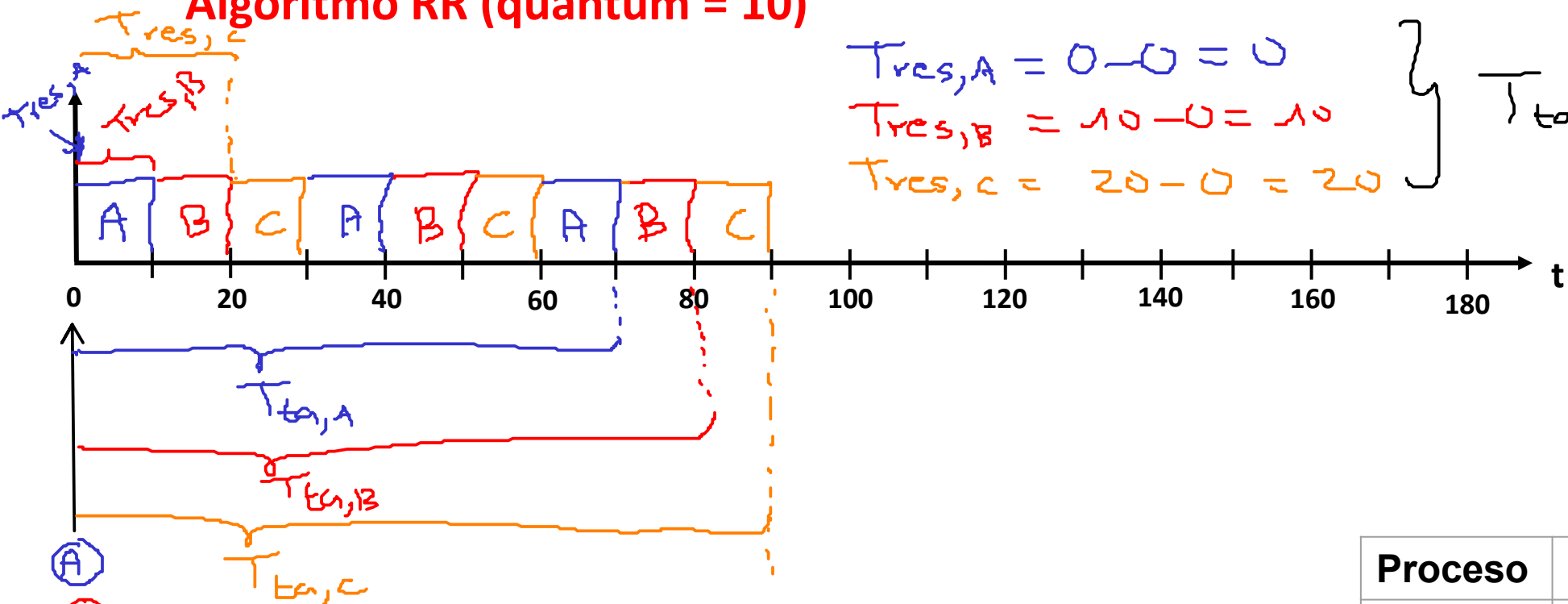


Proceso	Arrival time	Run-time
A	0	30
B	0	30
C	0	30



Round Robin (RR)

Algoritmo RR (quantum = 10)



$$T_{ta} = \frac{0 + 10 + 20}{3} = 10$$

$$T_{ta,avg} = \frac{240}{3} = 80$$

Proceso	T_{ta}	T_{res}
A	70	0
B	80	10
C	90	20
avg	80	10

Round Robin (RR) .vs. Shortest job first (SJF)

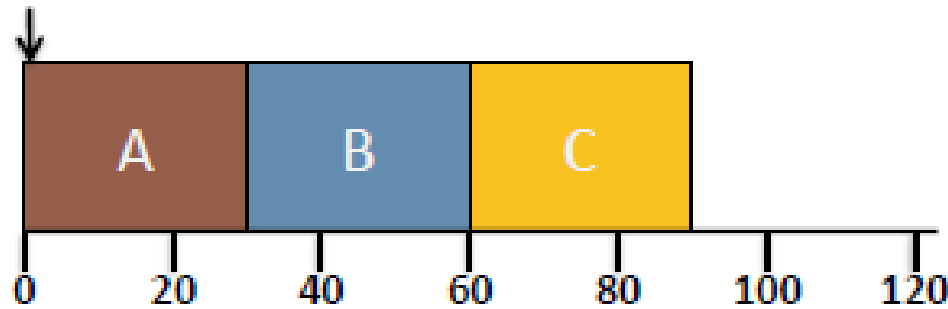
Comparación de los algoritmos

Suponga que se tiene una situación en la que:

- Tres procesos llegan a un sistema (A, B y C)
- Llegan en el tiempo 0 en el orden: A - B - C
- El tiempo de ejecución de cada uno de los procesos es de 30 seg.

Proceso	Arrival time	Run-time
A	0	30
B	0	30
C	0	30

A(30), B(30), C(30)



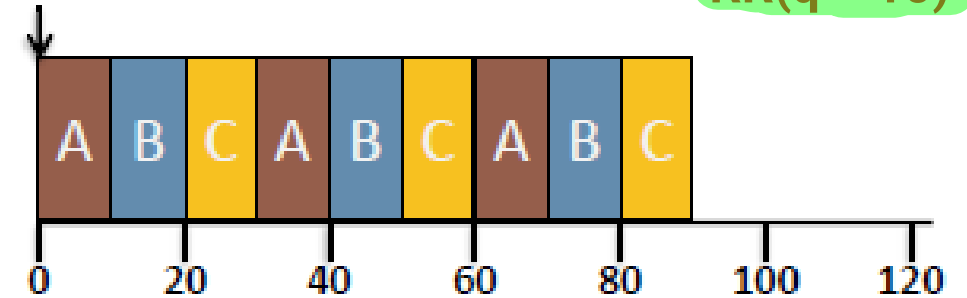
$$T_{\text{turnaround}} = (30 + 60 + 90)/3 = 60$$

$$T_{\text{response}} = (0 + 30 + 60)/3 = 30$$

SJF

* A(30), B(30), C(30)

RR(q = 10)



$$T_{\text{turnaround}} = (70 + 80 + 90)/3 = 80$$

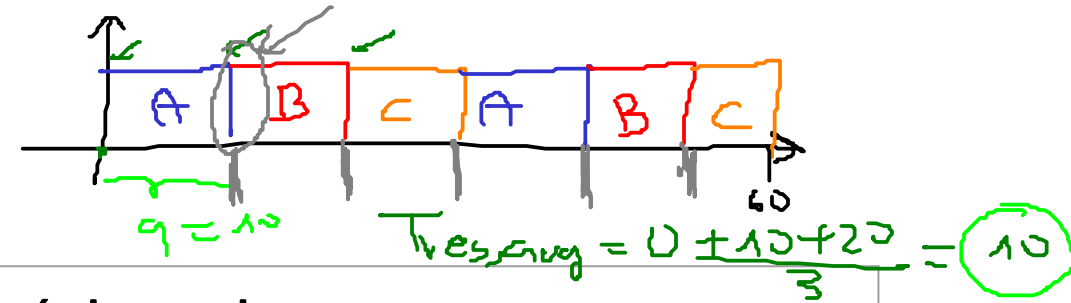
$$T_{\text{response}} = (0 + 10 + 20)/3 = 10$$

Mejor
mejor

Típicamente, el RR tiene un turnaround time más alto que el SJF (y similares), pero tiene un mejor tiempo de respuesta.

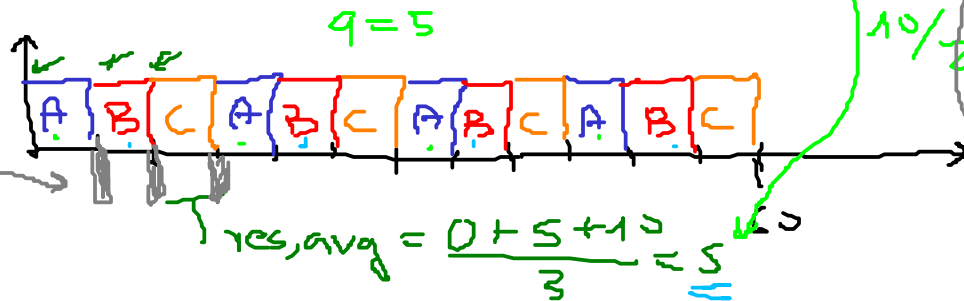
Round Robin (RR)

Importancia de la duración del quantum



Entre más corto el quantum

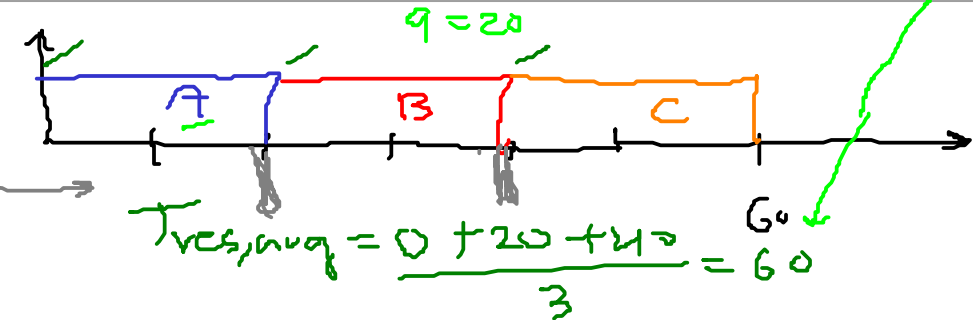
- **Mejor** tiempo de respuesta.
- El costo del cambio de contexto (context switch) **dominará** el desempeño global.



Banco

Entre más largo el quantum

- **Disminuye** el costo total del cambio de contexto.
- **Deteriora** el tiempo de respuesta.



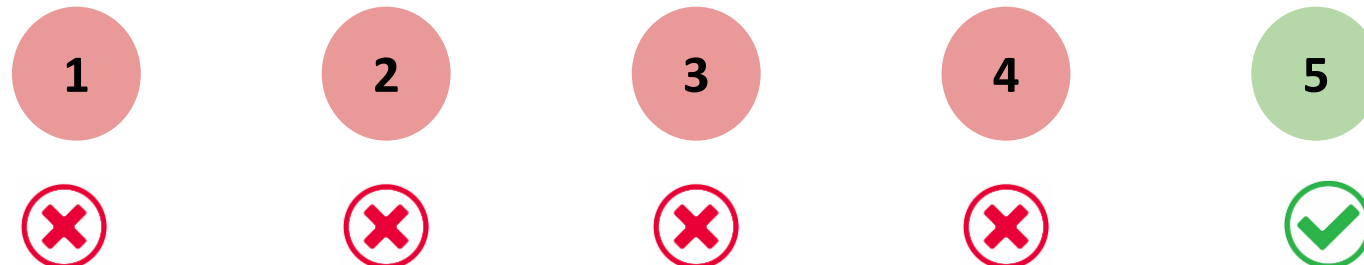
10×2

Decidir la longitud de la porción de tiempo es un **compromiso** que debe contemplar el **diseñador del sistema**.

Relajando las suposiciones ideales

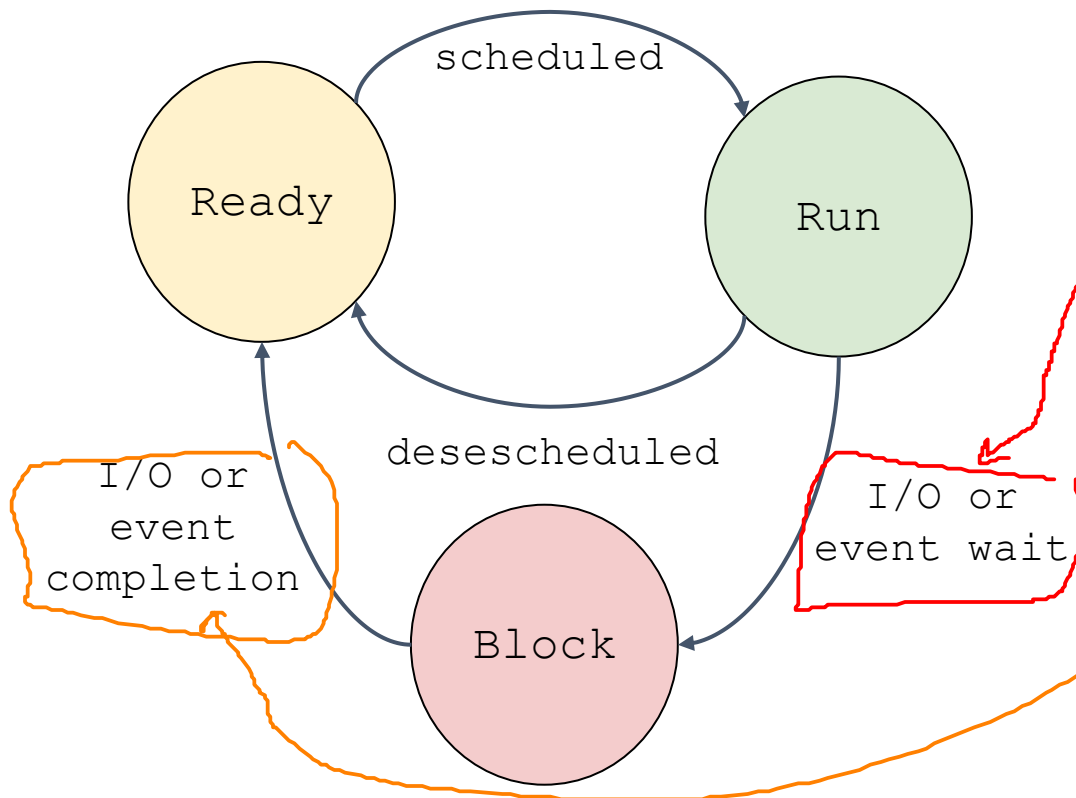
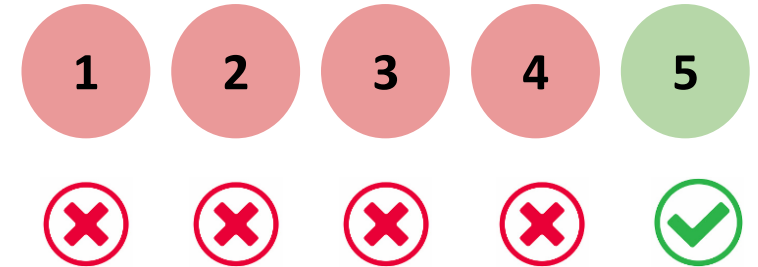
¿Que pasa si hay operaciones de I/O?

- ~~1. Cada trabajo se ejecuta por la misma cantidad de tiempo.~~
- ~~2. Todos los trabajos son iniciados al mismo tiempo (arrival time).~~
- ~~3. Una vez iniciado, cada trabajo se ejecuta hasta su finalización.~~
- ~~4. Todos los trabajos solo usan la CPU (no I/O).~~
5. El tiempo de ejecución de cada trabajo es conocido (runtime).



Incorporando I/O

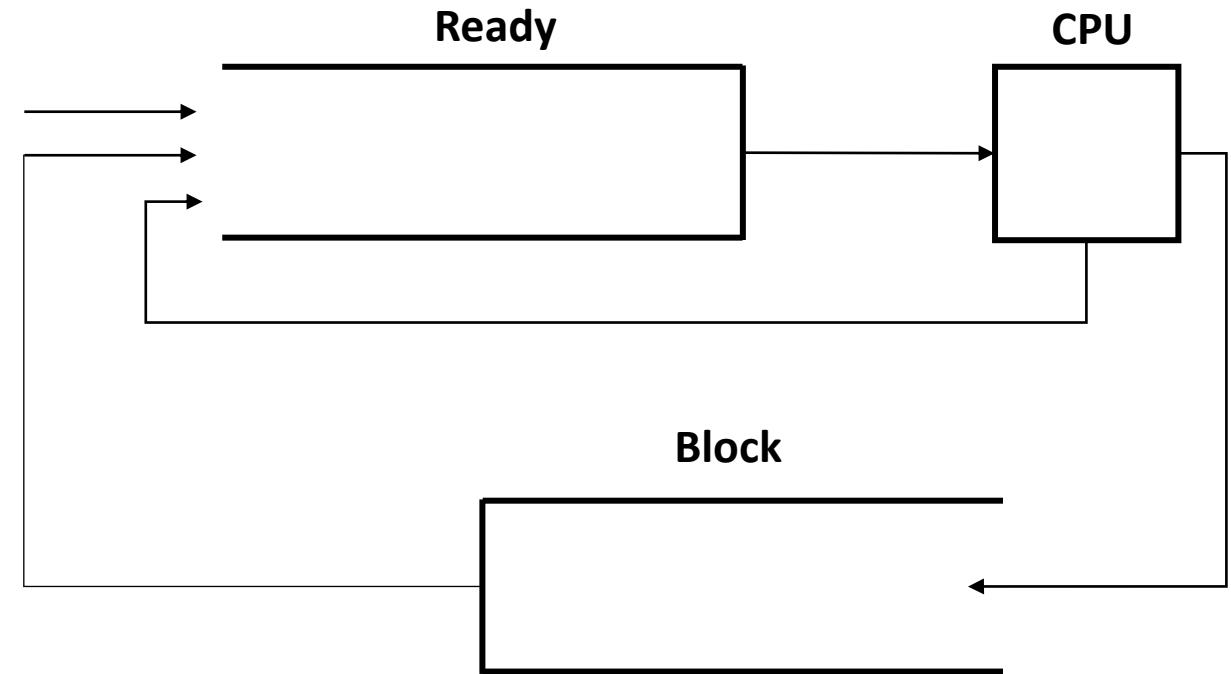
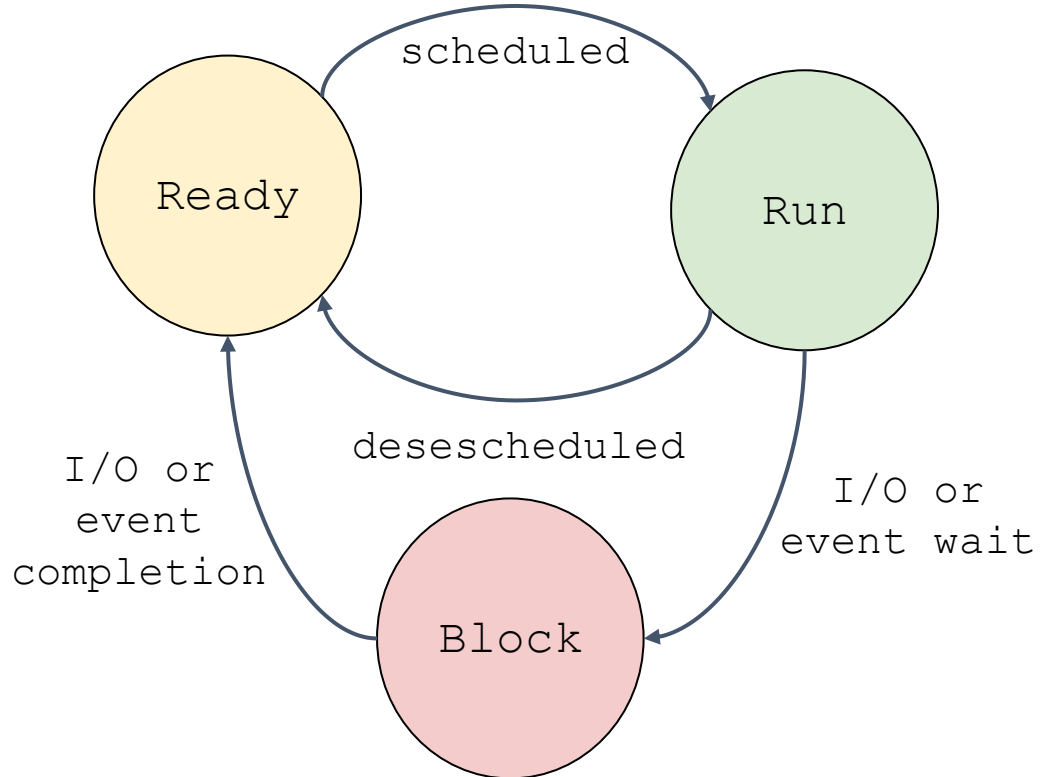
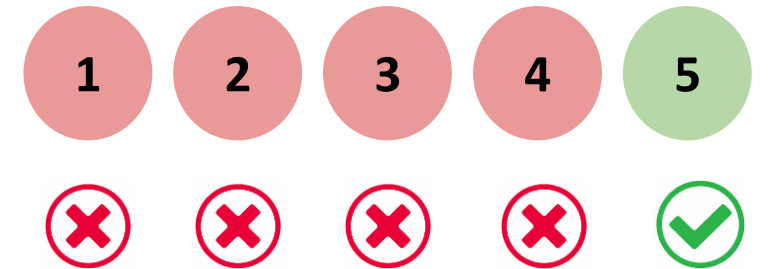
¿Que pasa si hay operaciones de I/O?



- Cuando un trabajo inicia un requerimiento de I/O:
 - El trabajo es **bloqueado** en espera de la finalización de la operación I/O.
 - El **scheduler** debería **seleccionar otro trabajo** para que tome uso de la CPU.
- Cuando la operación de I/O finaliza:
 - Se lanza una interrupción.
 - El SO mueve el proceso del estado bloqueado (**blocked**) al estado listo (**ready**).

Incorporando I/O

¿Que pasa si hay operaciones de I/O?

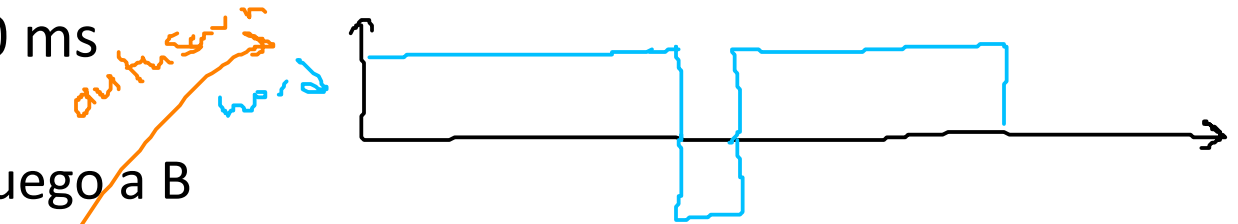


Incorporando I/O

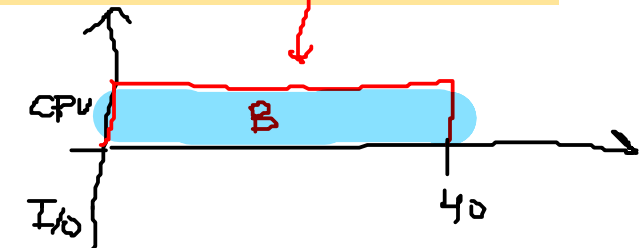
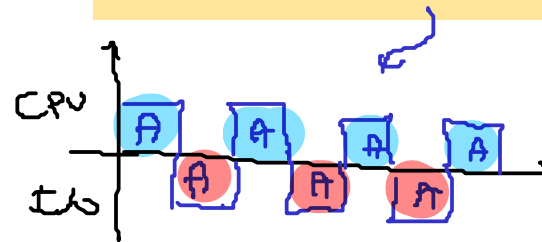
Ejemplo: Suponga que se tiene una situación en la que:

- A y B requieren 40 ms de CPU cada uno
- A se ejecuta por 10 ms y luego emite una solicitud de I/O
 - Cada operación de I/O toma 10 ms
- B no realiza operaciones de I/O
- El planificador ejecuta a A primero y luego a B

Proceso	Características
A	• CPU: 40 ✓ ◦ CPU burst: 10 • I/O: 30 ◦ I/O burst: 10
B	• CPU: 40 ✓



A (interactive) + B (CPU-intensive)

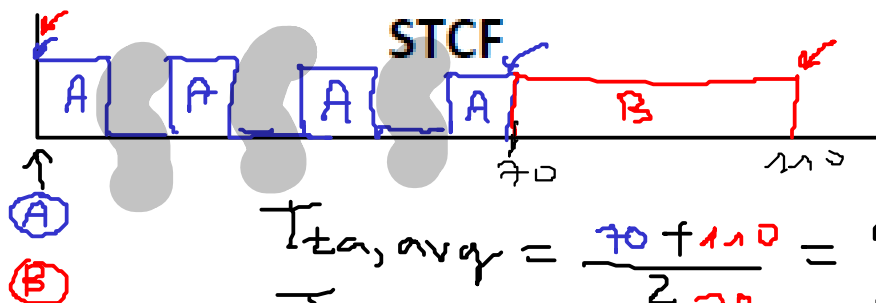
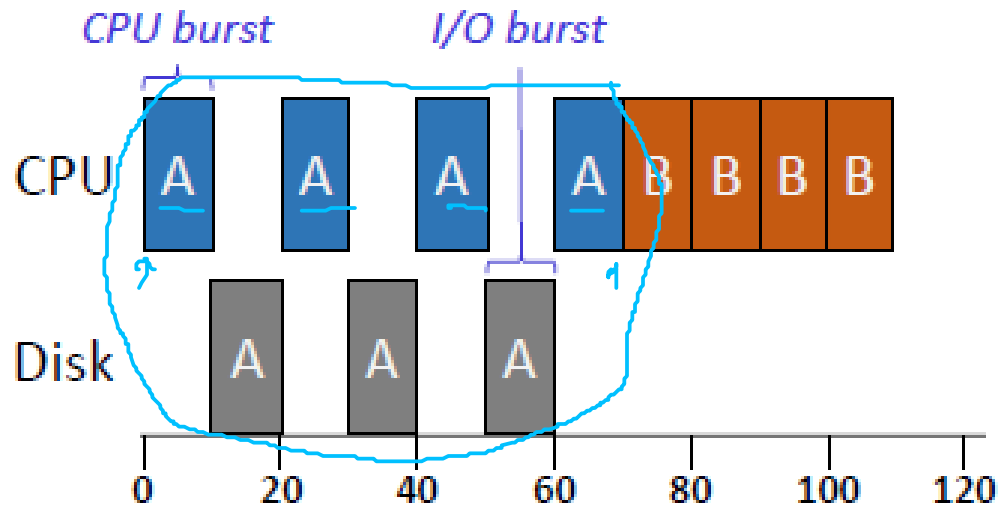


Incorporando I/O

Not I/O aware

SJF
40

Pobre uso de los recursos.



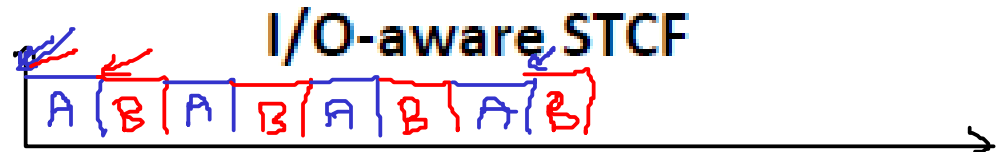
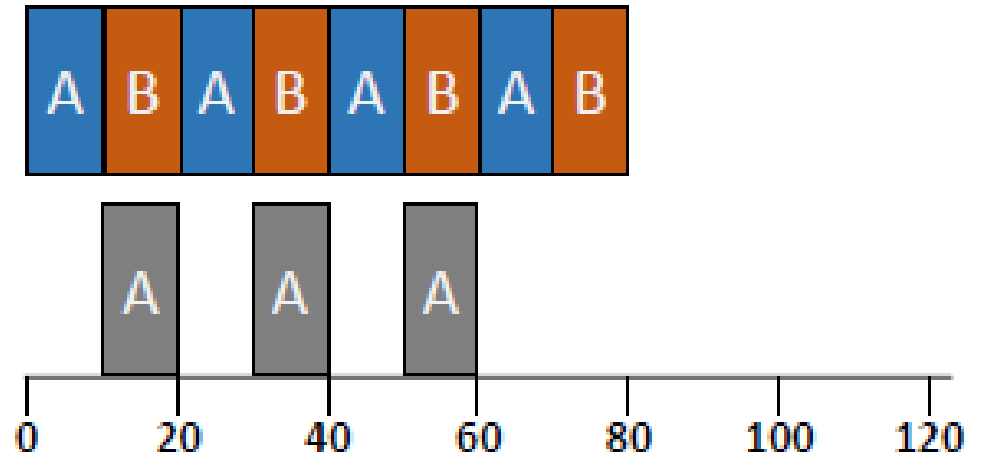
$$T_{ta,avg} = \frac{70 + 110}{2} = 90$$

$$T_{res,avg} = \frac{0 + 70}{2} = 35$$

I/O aware (Overlap)

SJF

Overlap: Permite un mejor uso de los recursos.

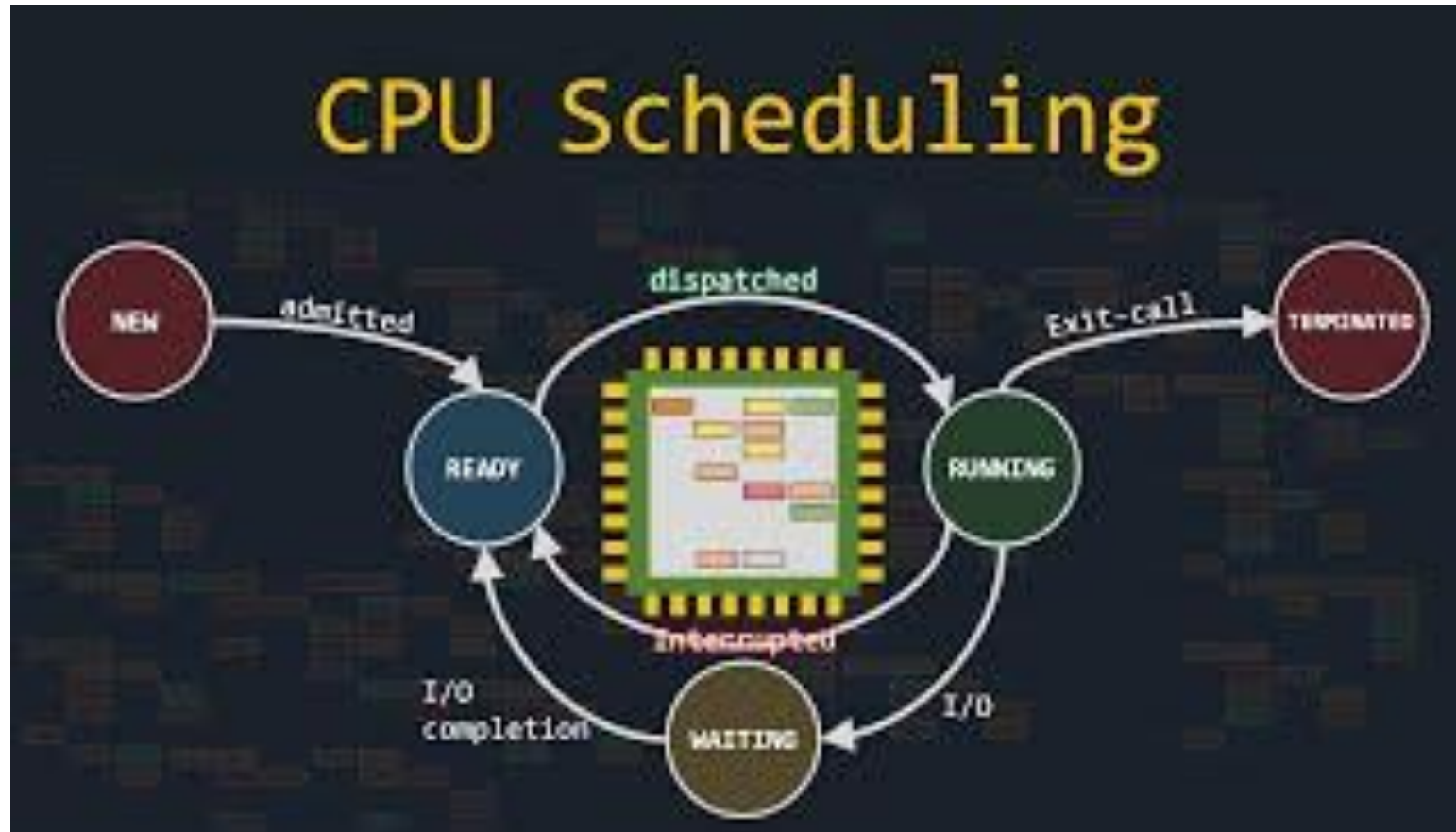


$$T_{ta,avg} = \frac{70 + 80}{2} = 75$$

$$T_{res,avg} = \frac{0 + 10}{2} = 5$$

Para comprender todo

Ahora que ha finalizado la clase, se recomienda que vea el video **The Fancy Algorithms That Make Your Computer Feel Smoother** ([link](#)) para que refuerce los conceptos anteriormente explicados.



Referencias

- **Operating Systems Three Easy Pieces** ([website](#): Capítulos [1](#) y [2](#))
- Videos de youtube: Clase 1 - Introducción a los sistemas operativos ([link](#)).
- Página de Remzi H. Arpaci-Dusseau ([link](#))
- Operating System Concepts - Tenth Edition ([website](#))
- Operating Systems Notes ([link](#))
- <https://github.com/Aniruddha-Tapas/Operating-Systems-Notes/blob/master/1-Overview.md>
- <https://myaut.github.io/dtrace-stap-book/kernel/proc.html>
- <https://myaut.github.io/dtrace-stap-book/index.html>
- <https://www.technologyuk.net/computing/computer-software/operating-systems/operating-system-utilities.shtml>
- <http://www.it.uu.se/education/course/homepage/os/vt18/module-4/simple-threads/>
- <https://courses.cs.duke.edu/spring19/compsci590.1/slides/>
- <https://os-lab-simulator.vercel.app/>
- <http://cpuburst.com/>
- <https://www.merlot.org/merlot/viewMaterial.htm?id=773407108>